

git and Github

Multi-Cloud DevOp – Phase 2

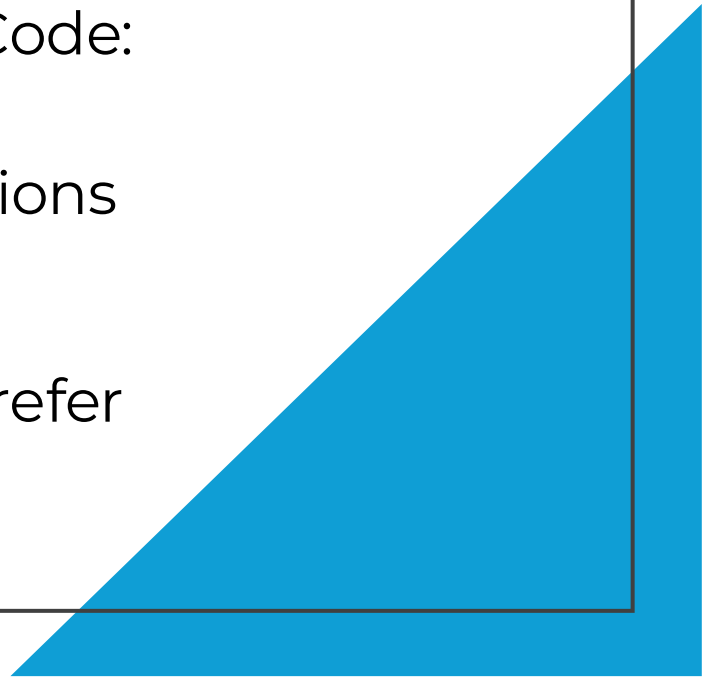
MacOs or Linux users

<https://git-scm.com/install/linux>

Text Editor

MacOS or Linux or Users:

- Our suggested text editor for this course is VS Code:
• <https://code.visualstudio.com/>
- It's created by Microsoft and has direct integrations with GitHub and is one of the most popular text editors today.
- You can follow along with any text editor you prefer however.



Windows Users

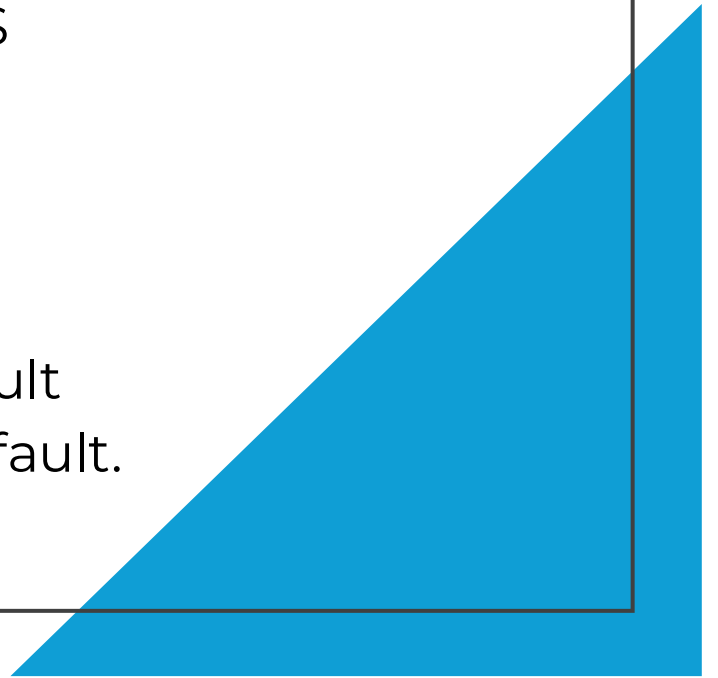
•Our *HIGHLY* recommend text editor for this course is VS Code:

•<https://code.visualstudio.com/>

•Why *HIGHLY* recommended?

•Windows + VS Code + GitHub

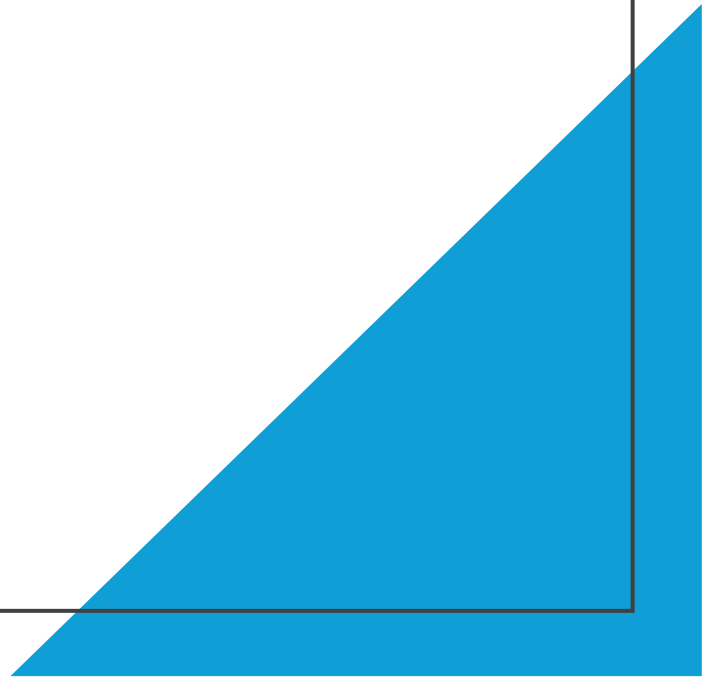
•Upon installing git you will be asked to select a default editor, you'll need VS Code installed to select it as default.



Download Git

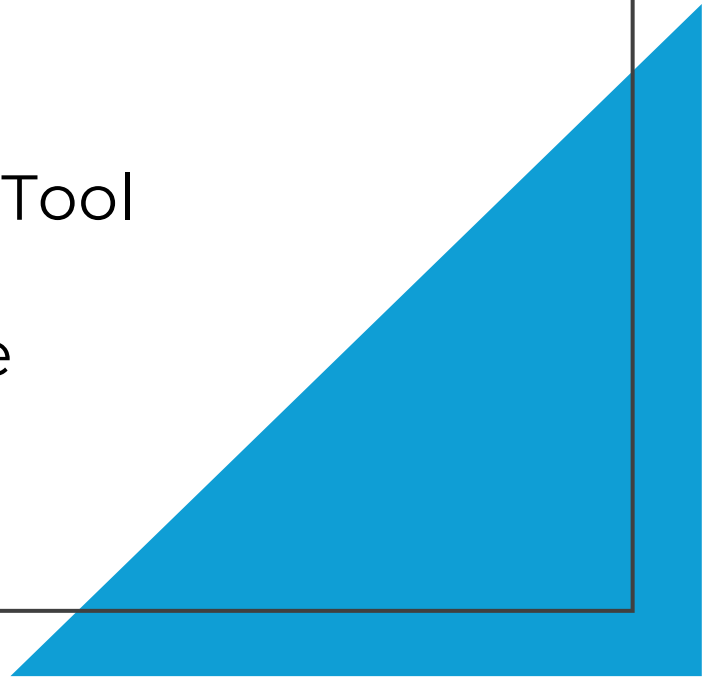
- **Windows Users:**

- Next we'll download git, go to:
 - **<https://git-scm.com/>**



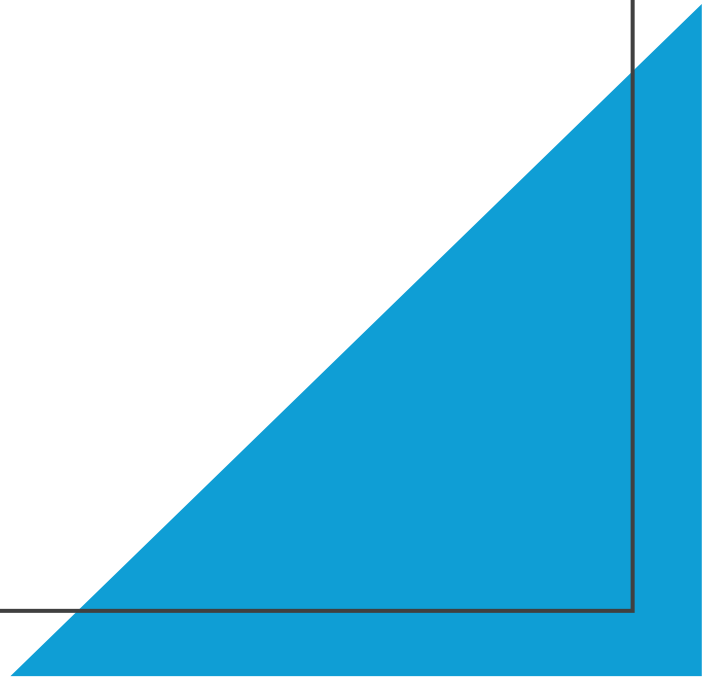
GitHub Profile Setup

- Now that we've downloaded Git, let's create a GitHub profile and also download the GitHub Desktop Tool for a GUI.
- We'll also show you how to set-up the VS Code GitHub extension.

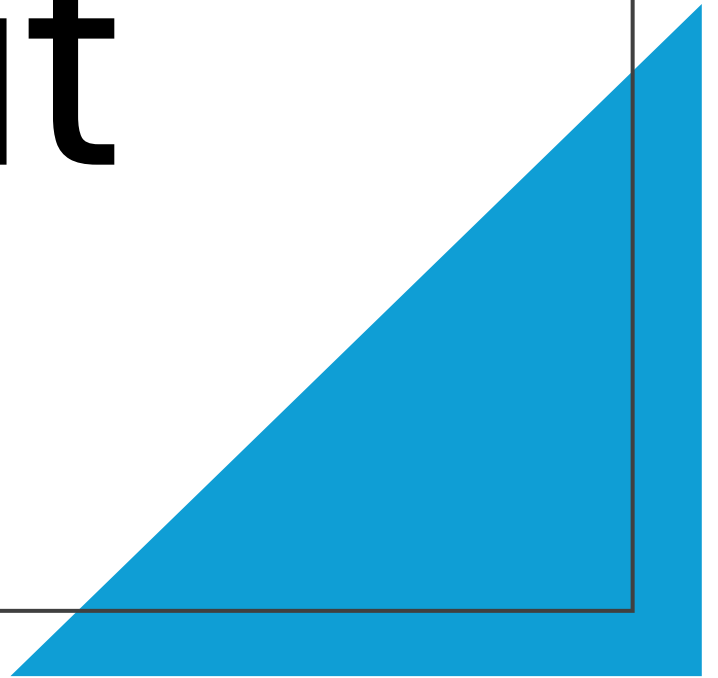


GitHub

- Take careful note of the user name and email address you register with at GitHub, ideally it will be the same username and email you configure git with locally.
- We will configure git in the next lecture.
- Let's create an account at:
 - www.github.com

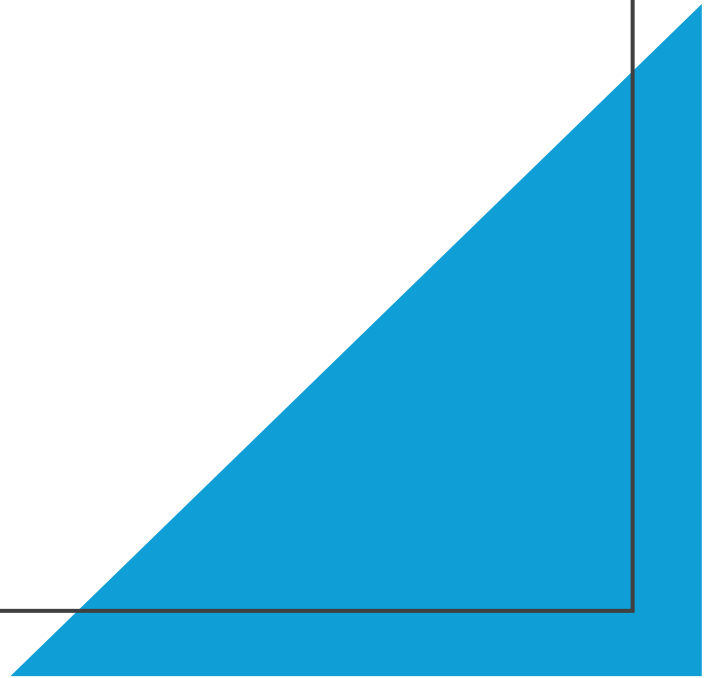


Configure Git



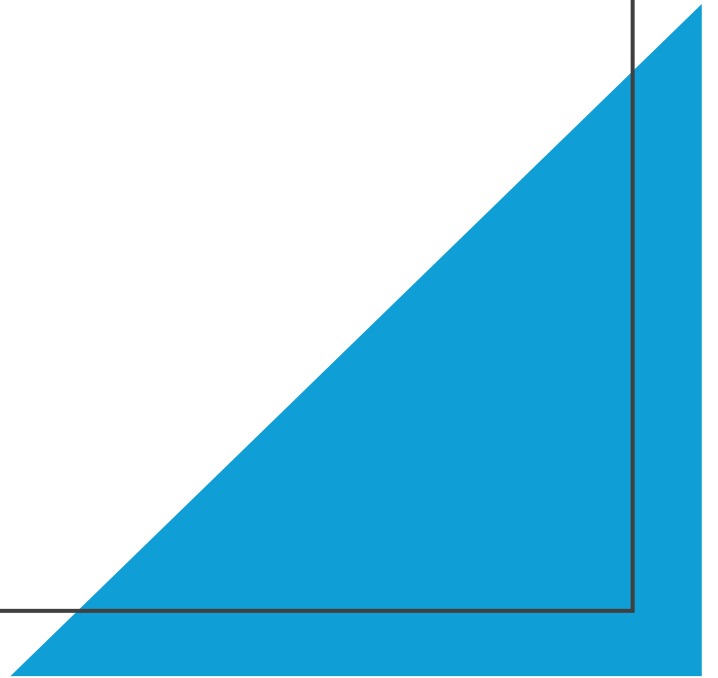
Introduction to Git

Day 2



Starting with Git

- Our main goal here is to understand Git, GitHub, and code repositories.
 - Understanding git and GitHub
 - Installing Git
 - Creating GitHub Profile
 - GitHub Tour
 - Creating and Cloning a git Repository
 - GitHub Desktop and Extensions
 - Exercise and Solution





Starting with Git

- Also just a quick note, you will see it stylized as **git** or **Git**.
- Git typically refers to the entire project, while git is the actual program used.

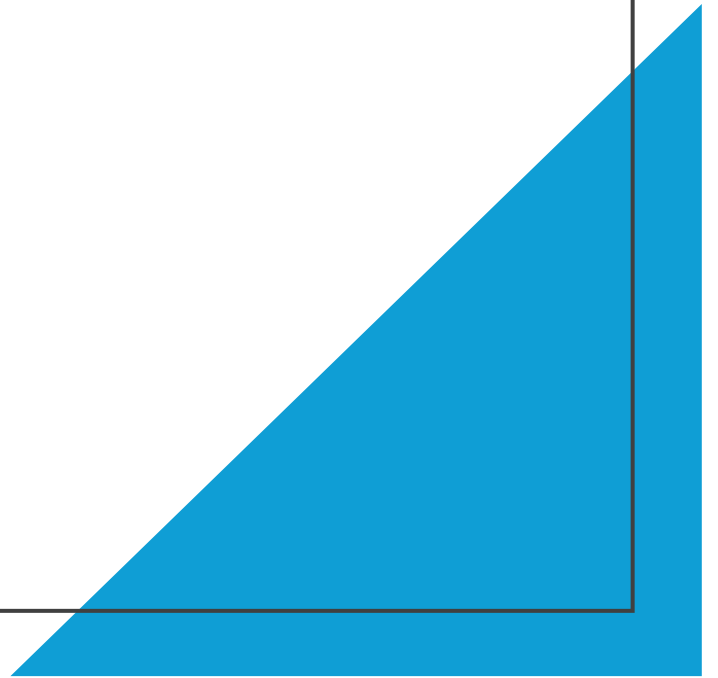
Starting with Git

- Linus developed the basic foundation of git in just 3 days!
- If you want to view the source code, Git itself is hosted on GitHub:
 - [**https://github.com/git/git**](https://github.com/git/git)



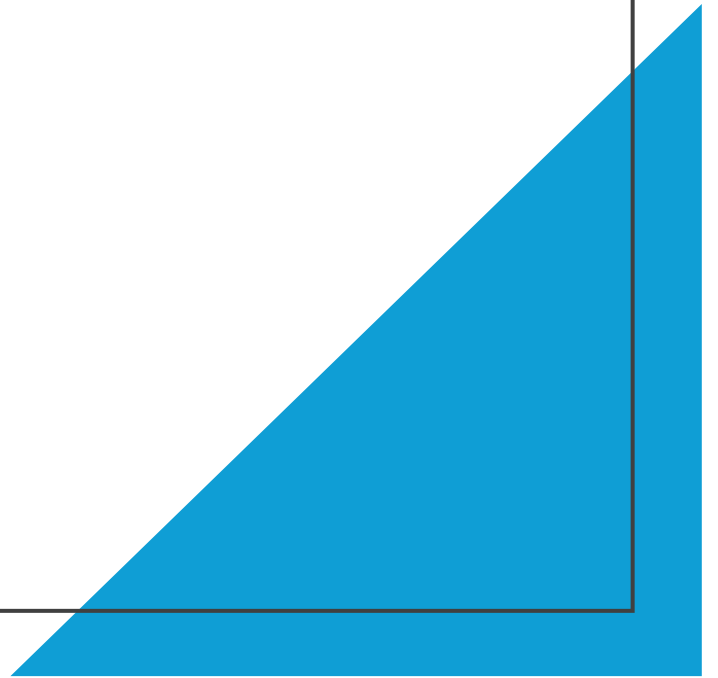
Starting with Git

- GitHub was started in 2007 as an internet hosting service for software development and version control using Git.
- While GitHub has many free tier features, including hosting of repositories, their paid features revolve around the needs of corporations, such as multiple people working on a private code repository.



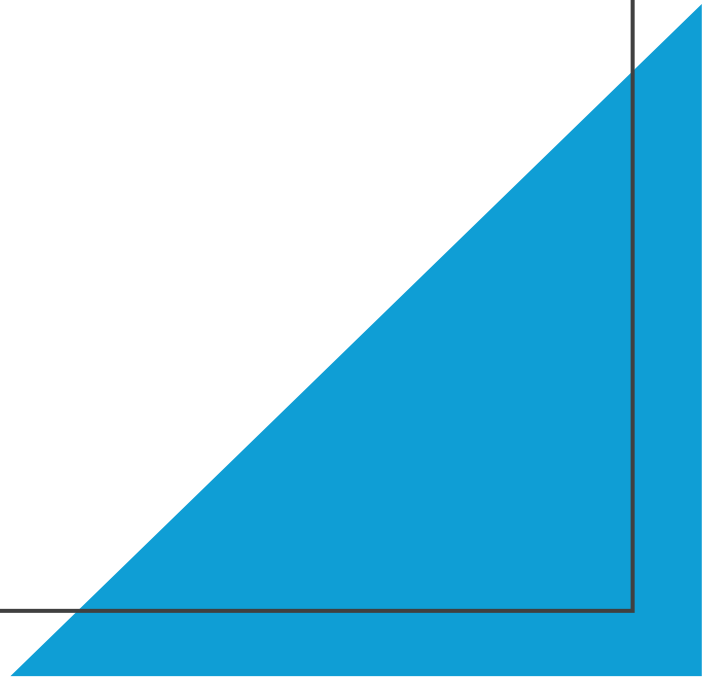
Starting with Git

- GitHub was acquired by Microsoft in 2018 for USD \$7.5 billion.
- Post acquisition there has been a deeper integration between GitHub and Microsoft products, including connecting to GitHub directly within VS Code, a code editor tool from Microsoft.



Starting with Git

- Git is the open source software that actually manages the git commands as a VCS.
- We will call it at the command line with commands such as: **git push**
- GitHub is the online hosting provider, that can act as a machine connected to our local machine via the internet and host code in a repository for us.



Starting with Git

- All the features and git methods shown in this course will fall under the free tier of GitHub.
- You should also know that you can now perform the commands with the GitHub Desktop Tool (a GUI for git commands) and the VS Code with GitHub extensions.
- There are actually many, many GUIs that have been created for Git, you can find the official list here:
 - <https://git-scm.com/downloads/guis>
- This course focuses on understanding how to use Git at the command line, since its still most commonly used that way, however we will also show you how to perform commands with GitHub Desktop.

GitHub Profile Setup & Github Desktop

- Now that we've downloaded Git, let's create a GitHub profile and also download the GitHub Desktop Tool for a GUI.
- Take careful note of the user name and email address you register with at GitHub, ideally it will be the same username and email you configure git with locally.

Starting with Git

- You can check the current configuration with the commands:

 - **git config user.name**

 - **git config user.email**

- The configuration commands will be:

 - **git config --global user.name "user"**

 - **git config --global user.email "email"**

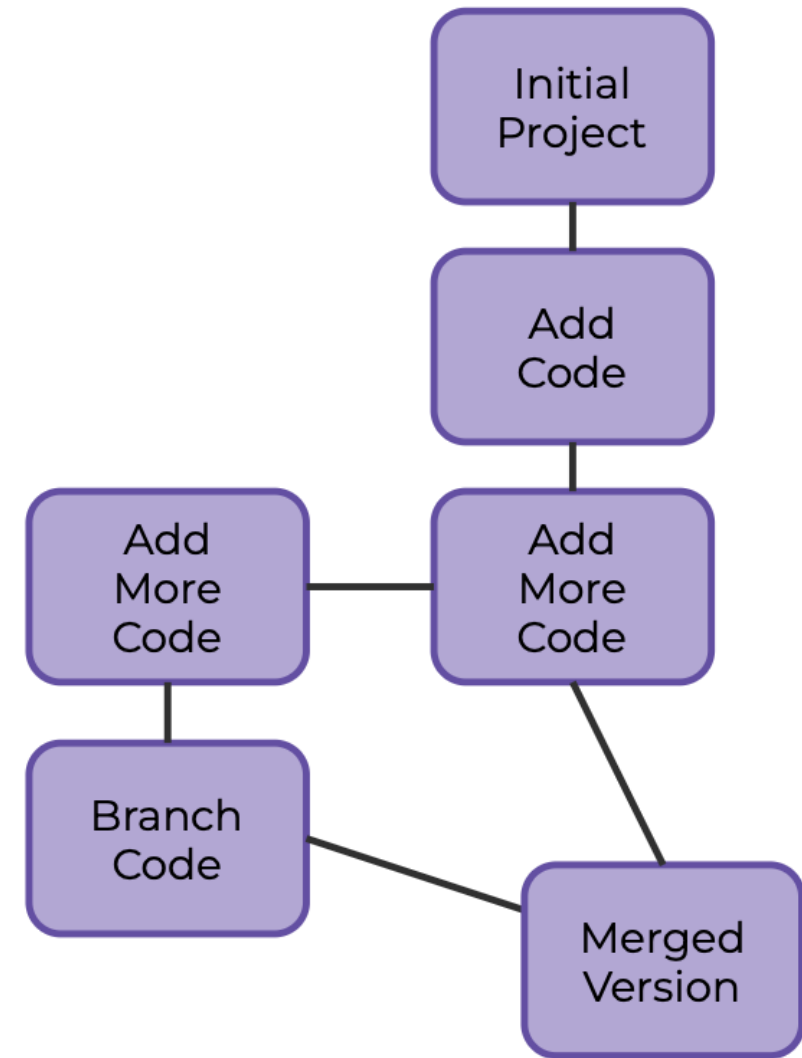


Starting with Git

- Let's head over to our command line interface to set-up our Git configuration:
 - Git Bash
 - Terminal
 - Command Prompt

Creating a Git Repository

- The main place we track changes and manage our files that are using Git is called a **repository**.
- Let's explore a public repository
 - Tensorflow

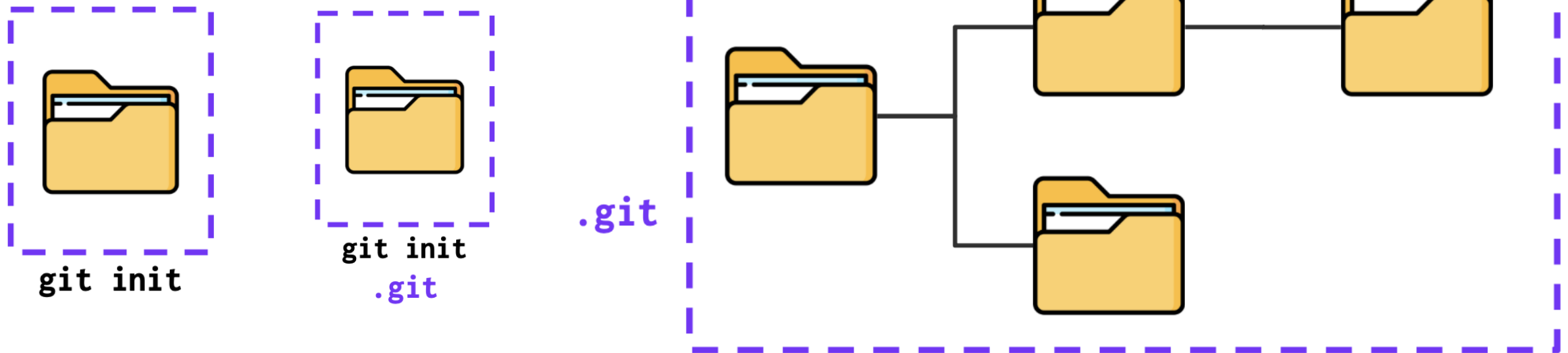


Creating a git repository

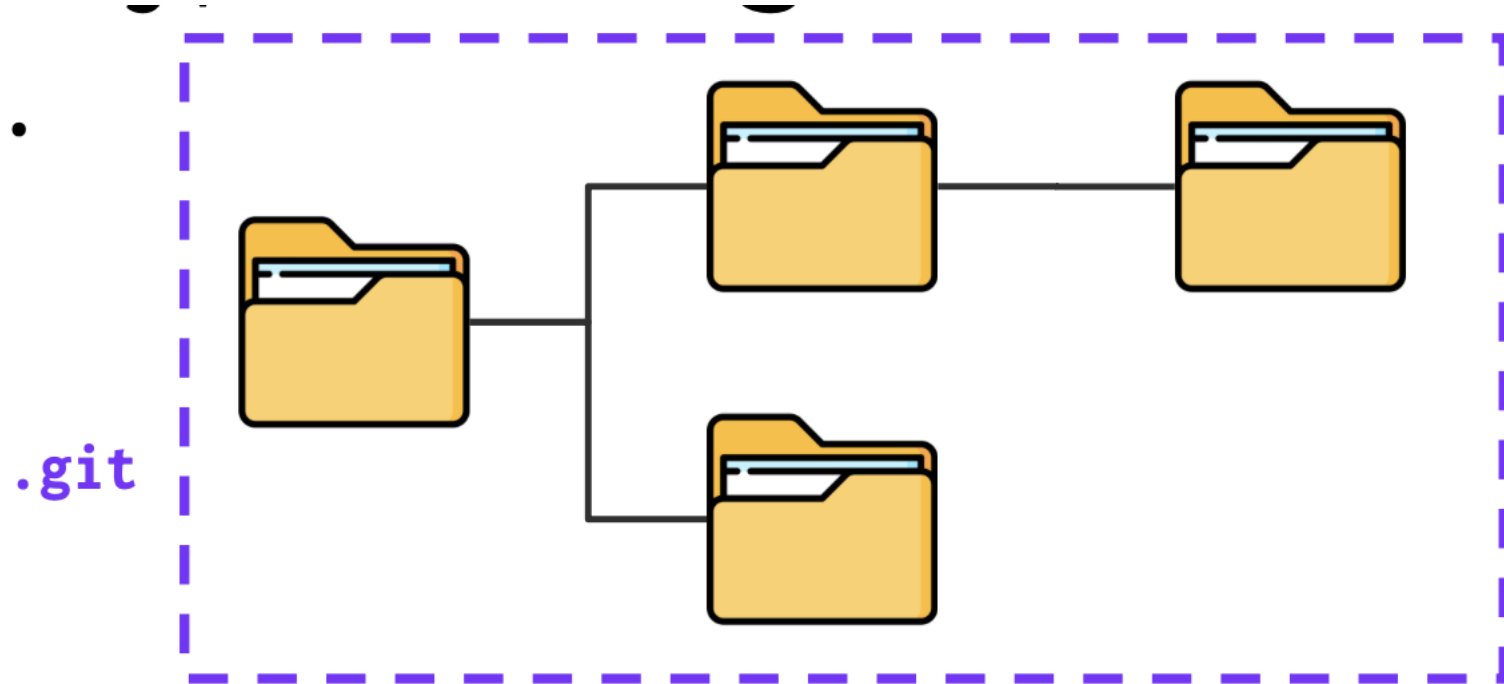
- How can we create a Git Repository?
 - **git init**
 - This command initializes a Git Repository on your local machine.
 - You only need to run this command once per project.
 - **git status**
 - This command will report back the status of your Git repository.
- Upon creating a repository with **git init** you will create a hidden .git file.
- The .git file is a hidden file that manages the versioning of the files inside the Git repository.

Git inside a Folder/Directory:

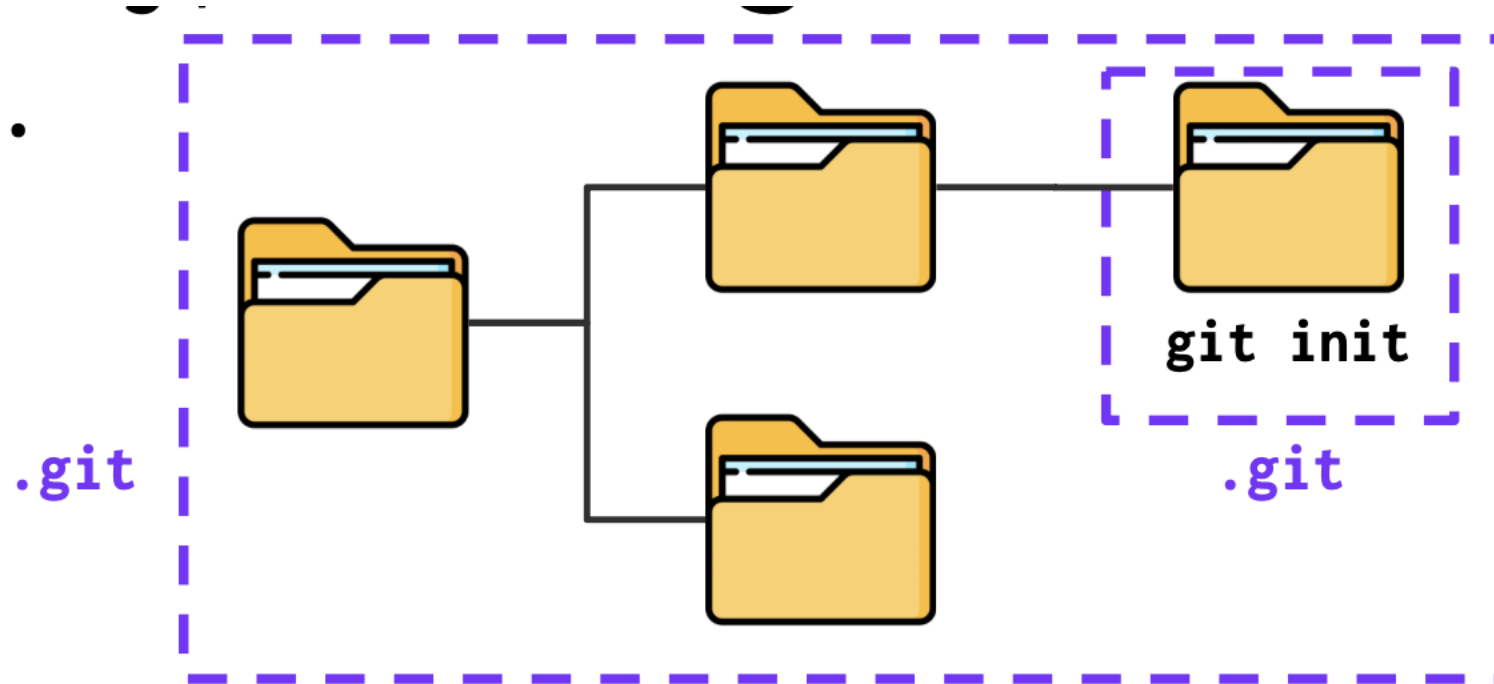
- Upon creating a Git Repository, all the folders/directories inside the top level Git Repository will also be part of that Repository, meaning all the changes are tracked.



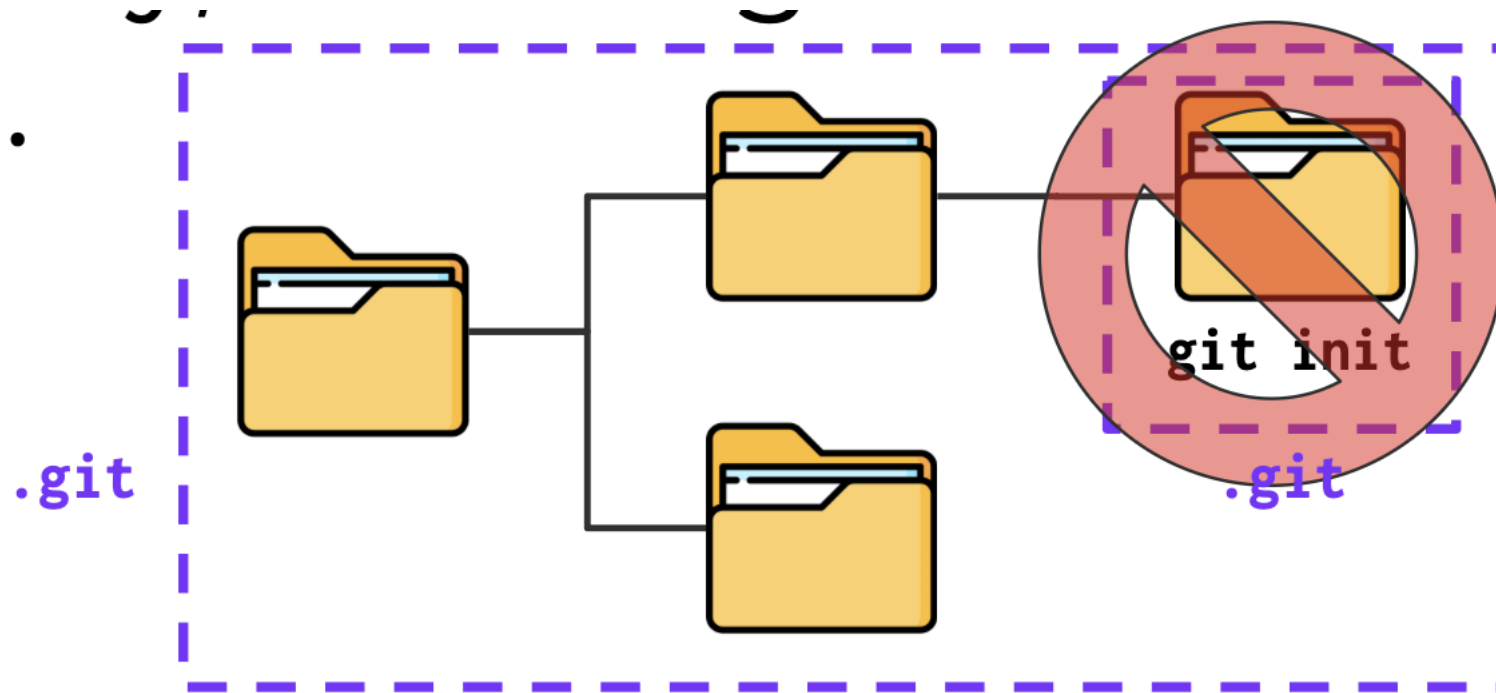
Git inside a Folder/Directory:



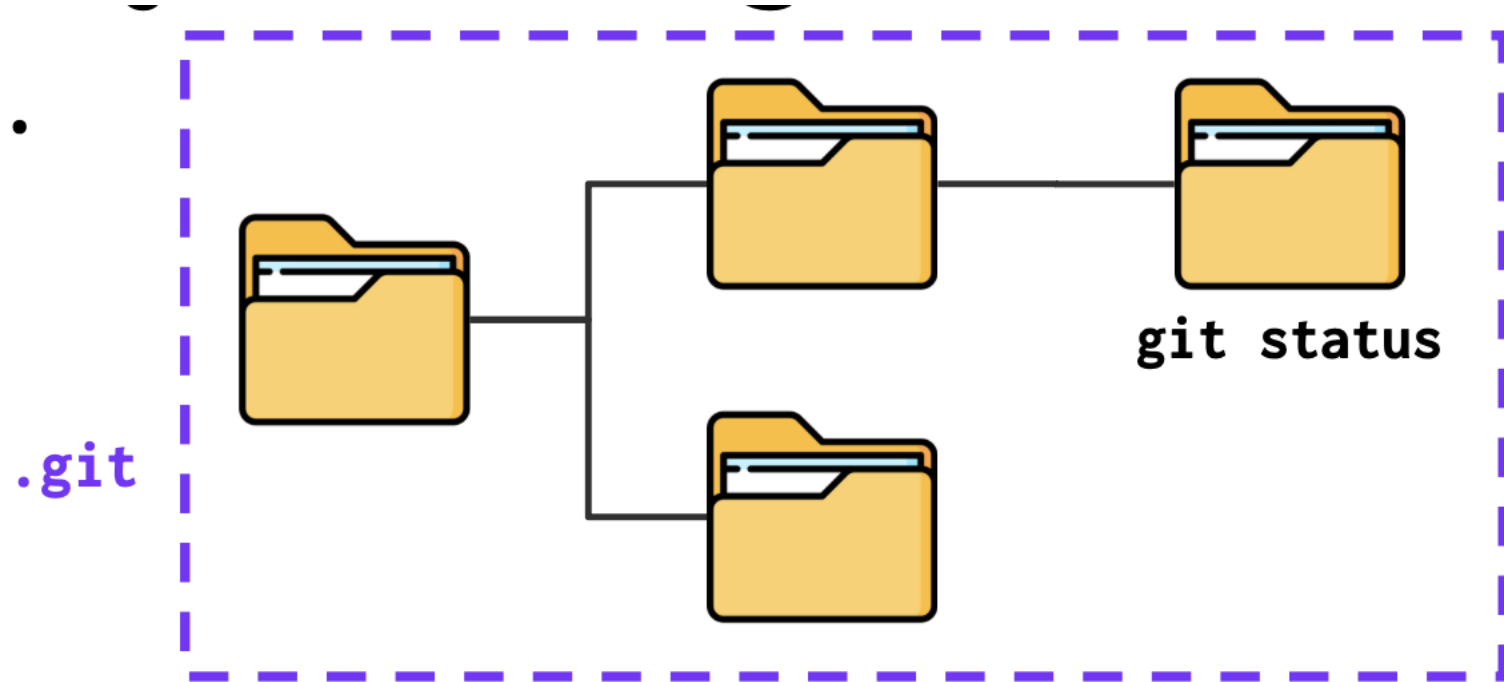
Git inside a Folder/Directory:



Git inside a Folder/Directory:



Git inside a Folder/Directory:



Creating Git repostory

- How can we create a Git Repository?
 - We can also use the Graphical Interface with GitHub Desktop or we can even create a new repository online at www.github.com.

Then we can **git clone** this repository to our local machine.

Creating Git repository

- Let's create our first local Git repository at the command line.
- Then we'll create a repository on GitHub and use **git clone** to clone it to our local computer.
- We'll need to set-up some tokens in order to clone private repositories.

Private Repositories and Tokens

We discovered we can easily clone other public repositories with the **git clone** command and then the HTTPS URL for the public repository.

Now let's explore how to deal with private repositories we wish to clone.

Cloning a private repository

- Option 1: Command Line:
 - Create Personal Access Tokens (PAT) on Github.com
 - When using the **git clone** command, reference the PAT.
- Option 2: GitHub Desktop Tool GUI:
 - Open the Github Desktop Tool
 - Login with GitHub Username and PW
 - Clone Repo via GUI

Cloning a git repository

- Clone Syntax with PAT:
- **git clone** <https://token@github.com/account/repo.git>
- Previously we used:
- **git clone** <https://github.com/account/repo.git>

Summary and Review

- Let's review the main Git and GitHub related methods we now know:

- **How to Create Repository**

- Locally via Command Line

- **git init**

- Online via GitHub.com

- Locally via GitHub Desktop Tool

Summary and Review

- Let's review the main Git and GitHub related methods we now know:

- **How to Clone a Repository**

- Public Repo from GitHub to our local machine via the Command Line

- **git clone**

- Private Repo from GitHub to our local machine via GUI and Command Line

Exercise Task

1

Create a new
Private
Repository on
GitHub.

2

Initialize your
repository with
README, license
and gitignore.

3

Clone your
Repository using
the Command
Line and a PAT.

Day 2

Starting to Use Git

Review:

- So far we've learned about version control, setting up Git and GitHub, and how to create Code Repositories.
- Now it's time to learn how to add code to these repositories, make changes or updates, and push or pull changes from repos to and from local machines.

Commands

Changing	Changing code in a Repository • git add
Committing	Committing these changes • git commit
Pushing or Pulling	Pushing or Pulling Changes • git push and git pull
Checking	Checking Status, Logs, and Changes • git status, git log, git diff

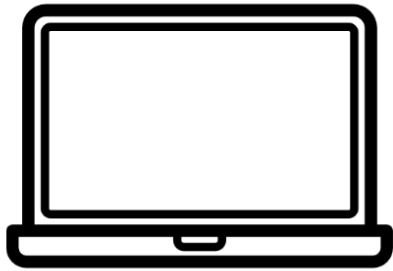
Basic Git Usage

Lets cover the basic cycle of a workflow using git and GitHub.

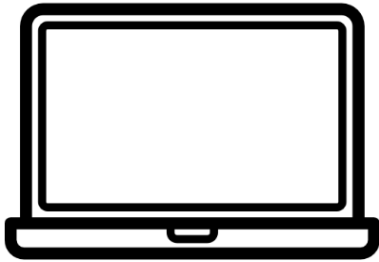
This particular basic example will assume you are just a solor developer and working on the same branch

We will cover branches and working with others later on

Workflow

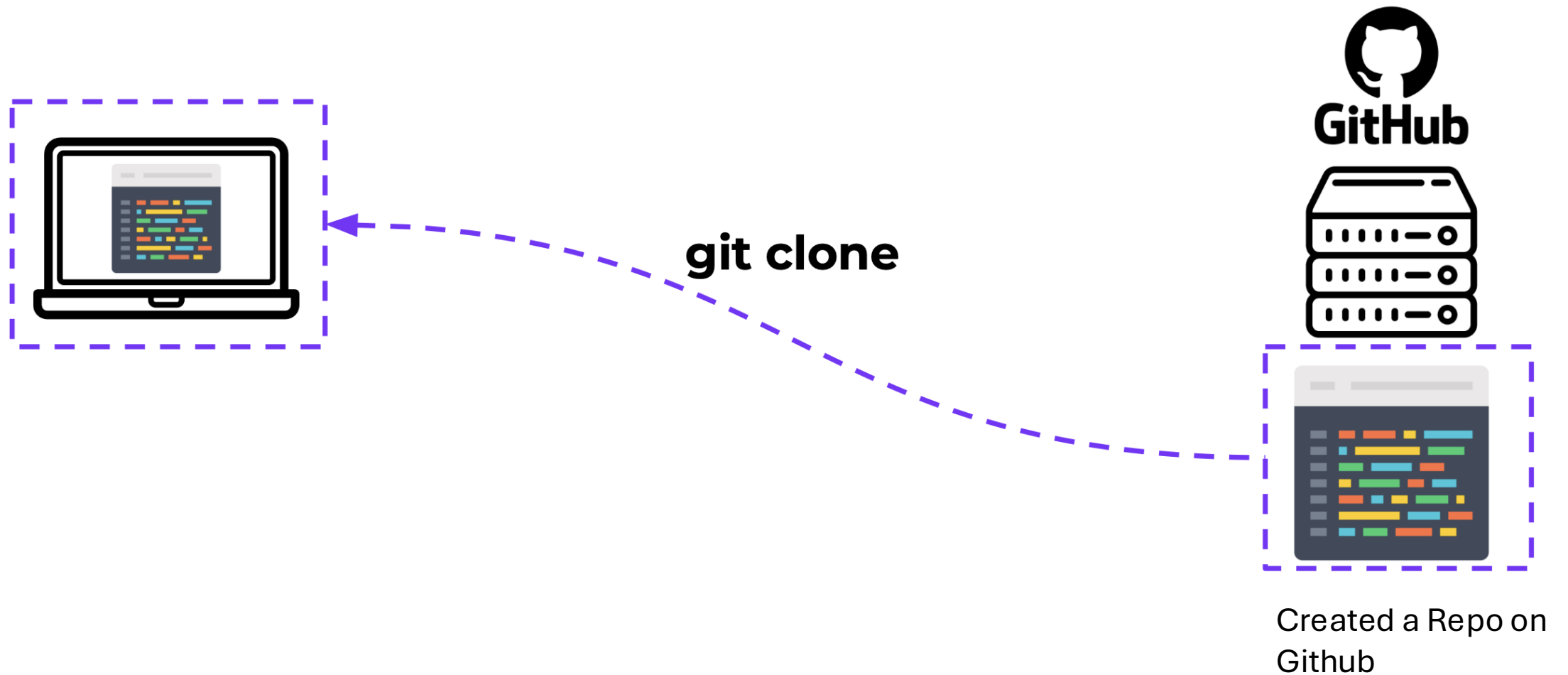


Workflow

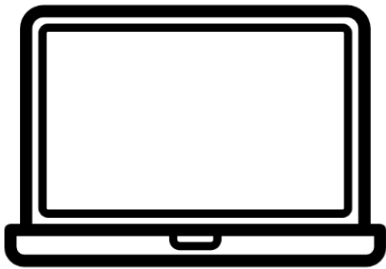


Create Git Repo
on GitHub

Workflow



Workflow



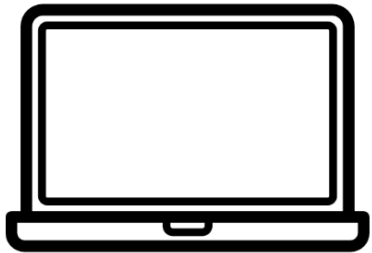
Workflow



Workflow



Workflow

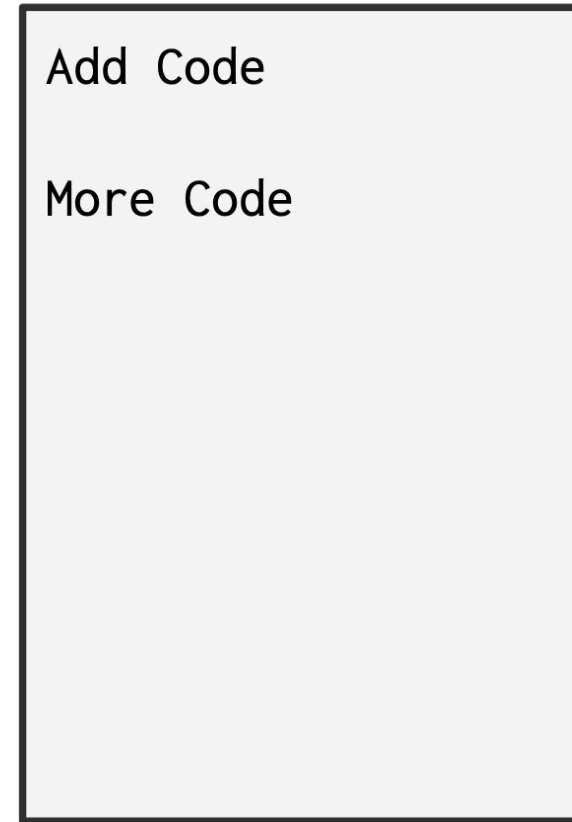
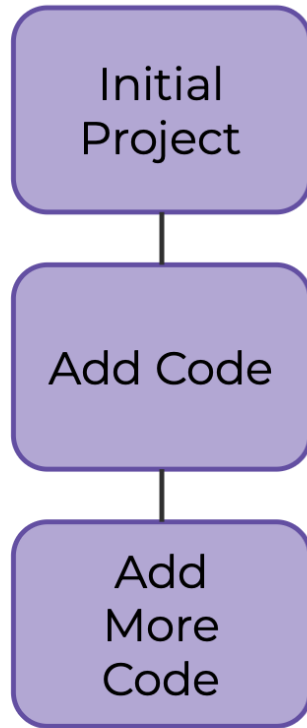


What we need to learn today:

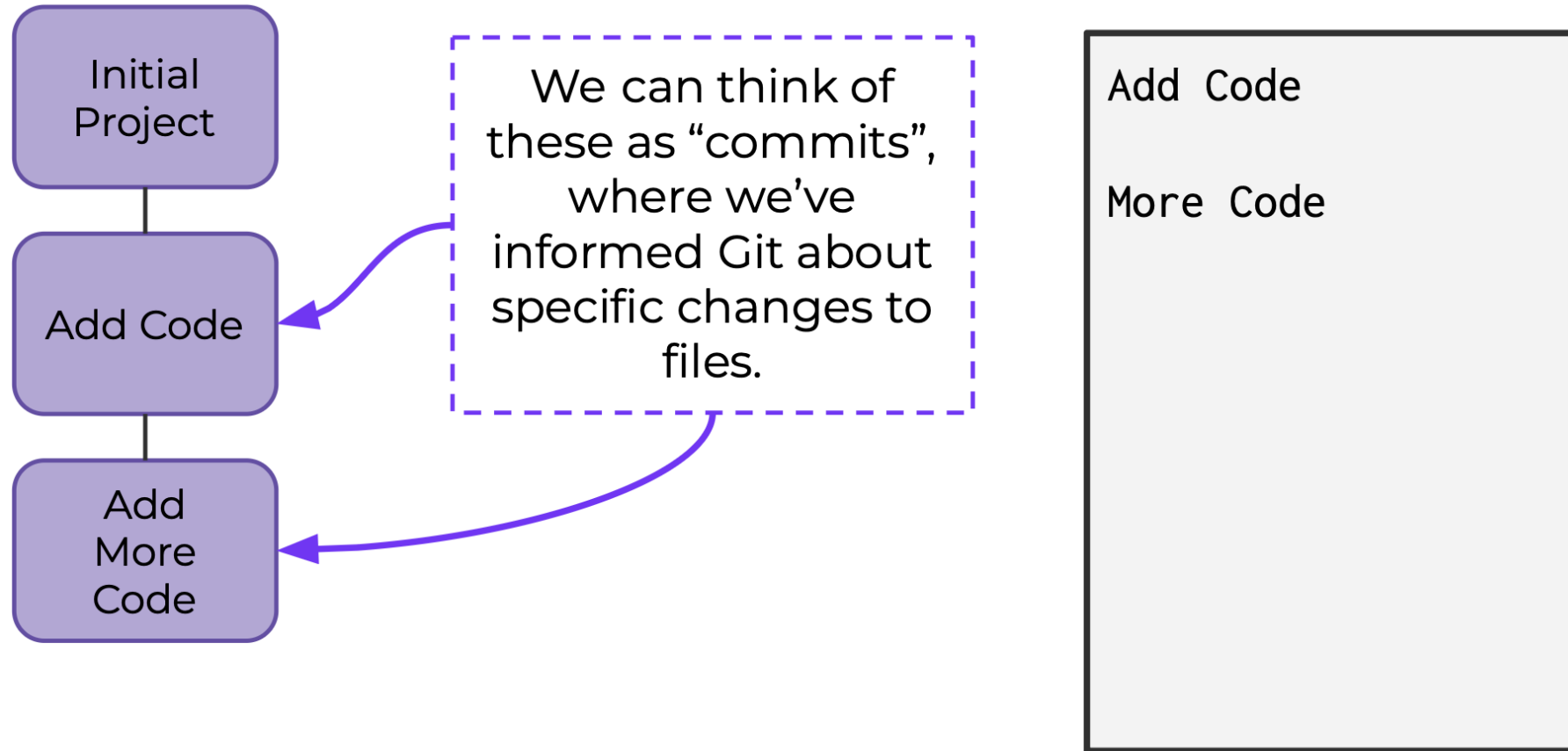
- Git Workflow
- How to tell Git about changes to our code
- How to push changes to GitHub
- How to pull changes from GitHub



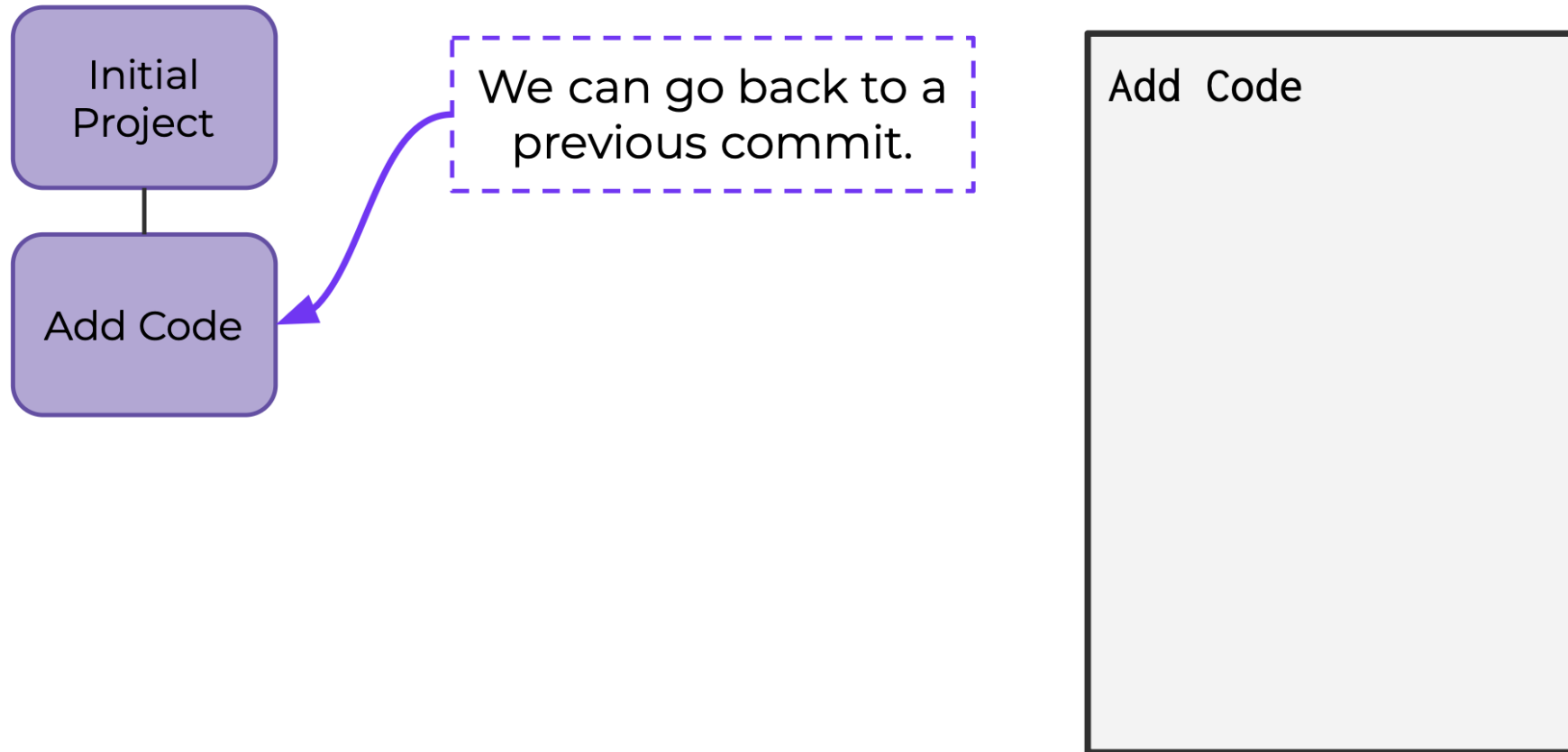
Workflow



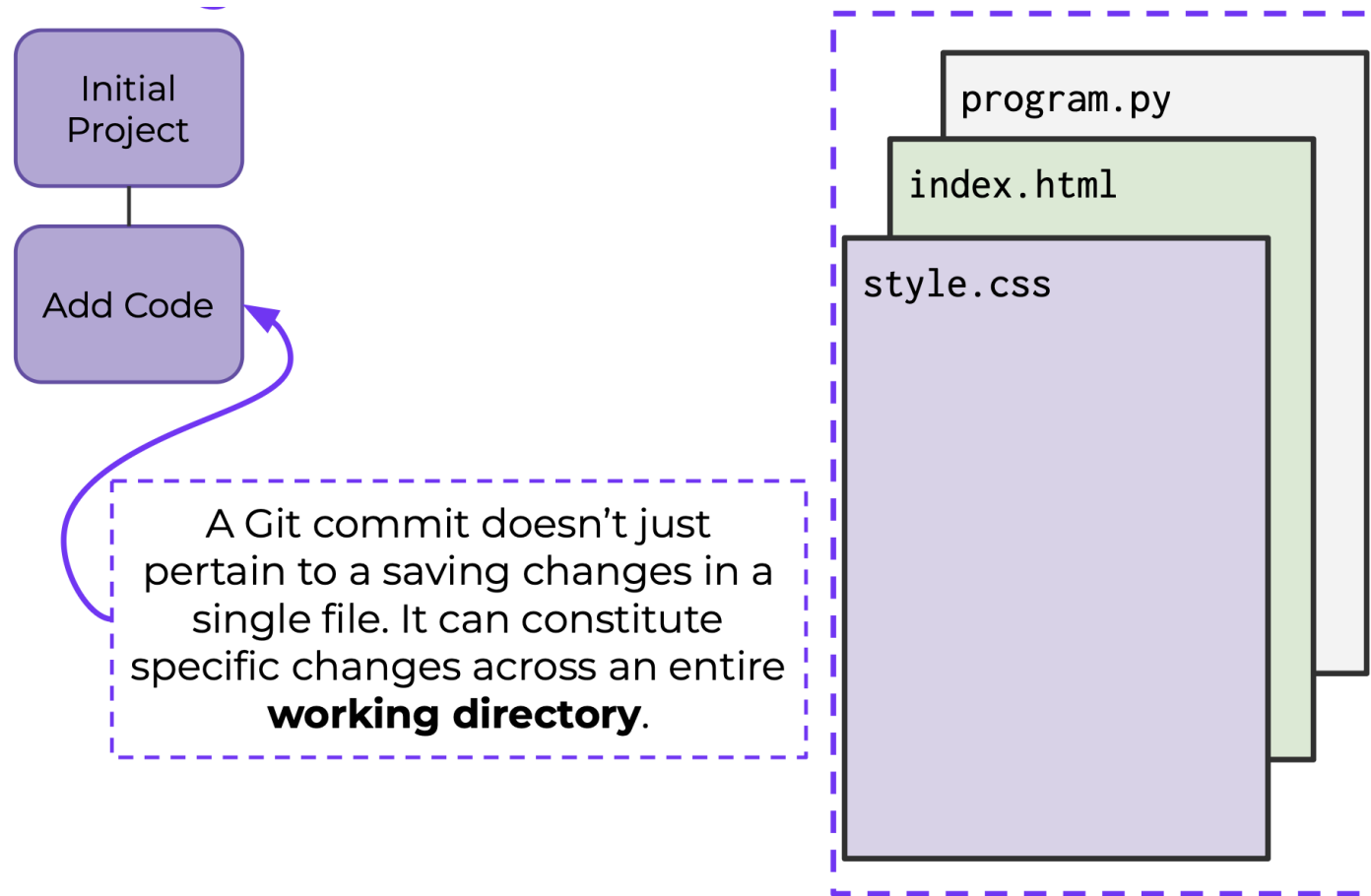
Workflow



Workflow



Workflow

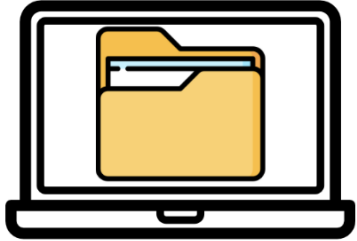


Workflow



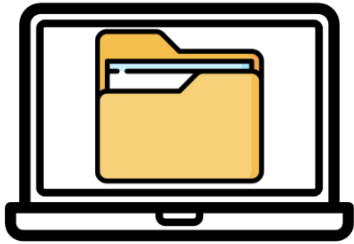
Workflow

Working Directory



Workflow

Working Directory



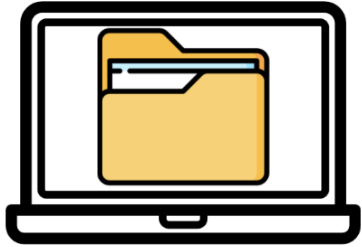
`program.py`

`index.html`

`style.css`

Workflow

Working Directory



`program.py`

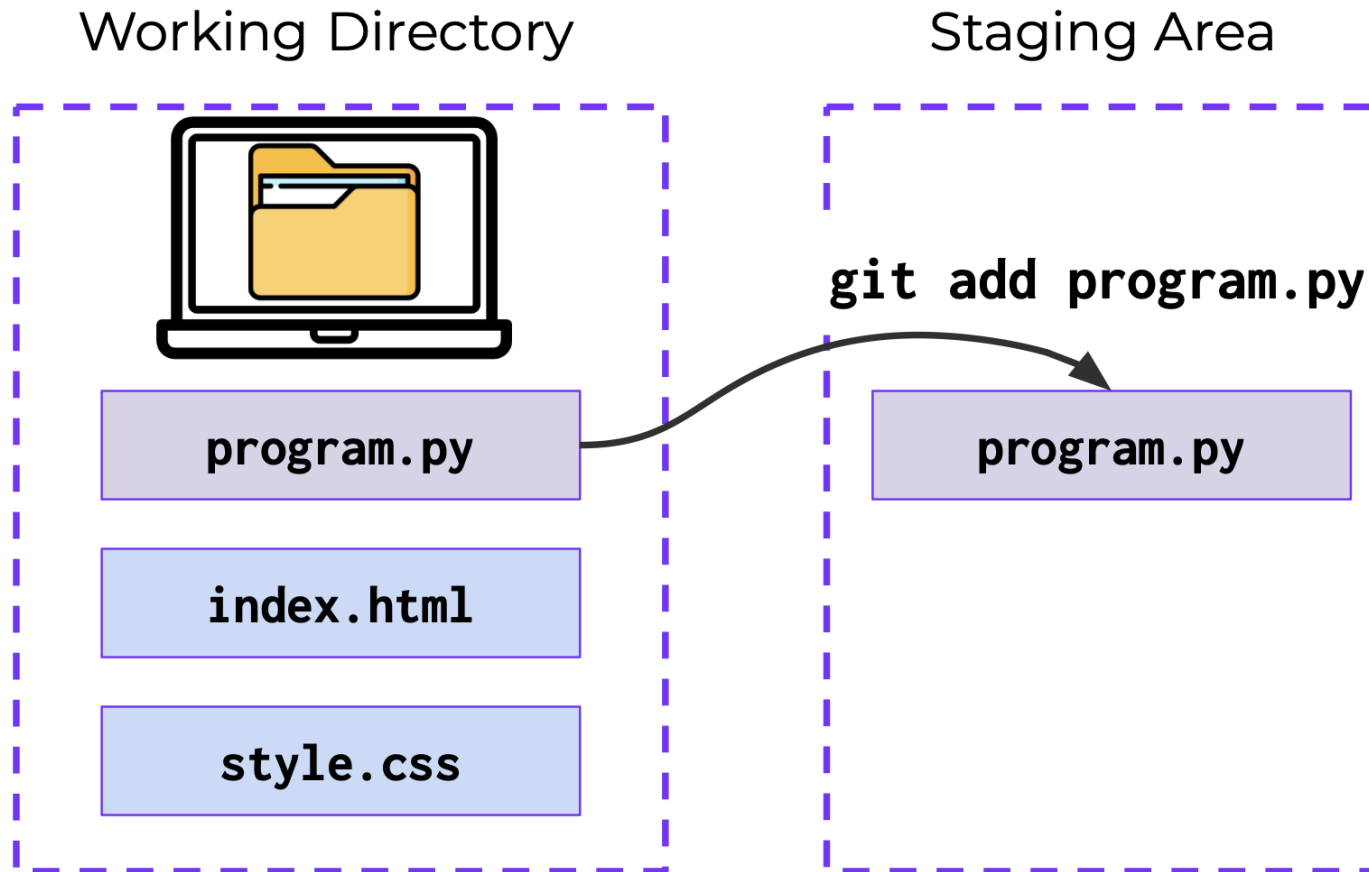
`index.html`

`style.css`

Staging Area

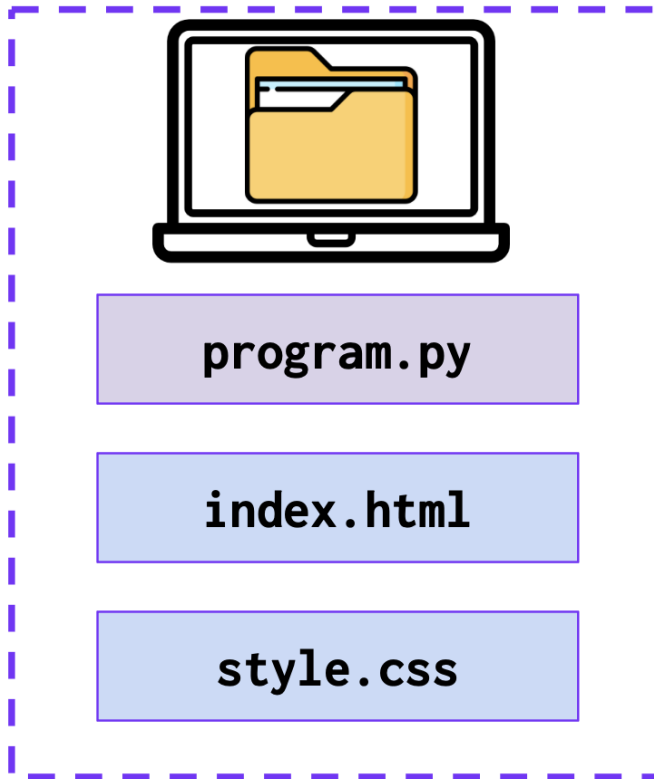


Workflow

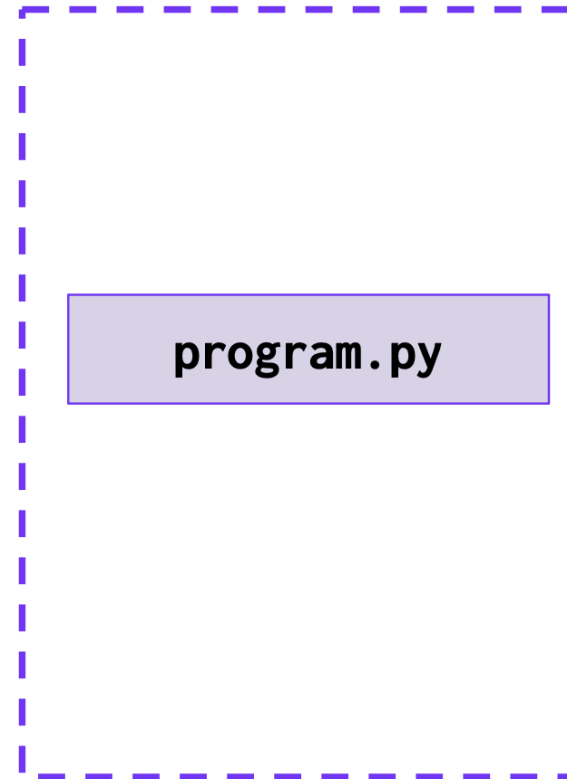


Workflow

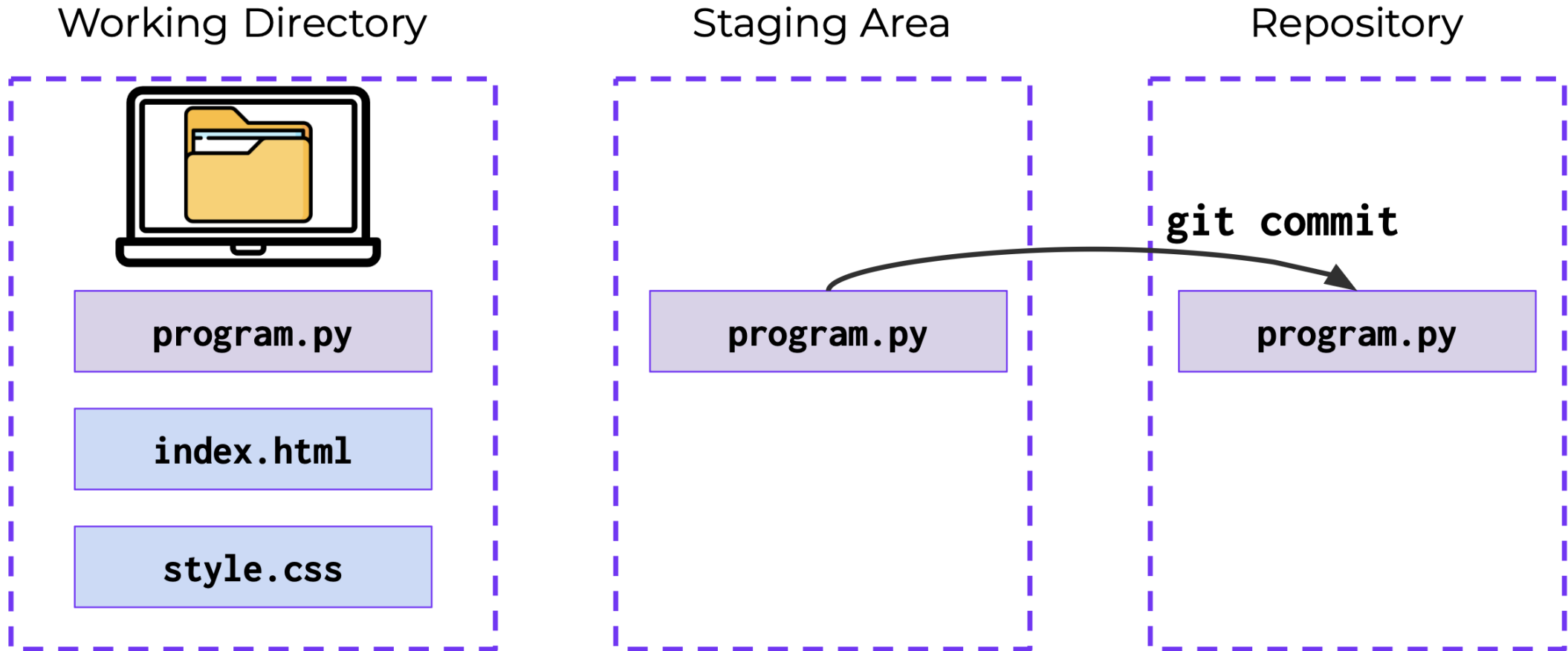
Working Directory



Staging Area

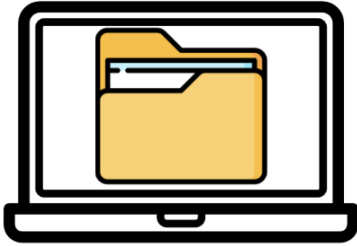


Workflow



Workflow

Working Directory



`program.py`

`index.html`

`style.css`

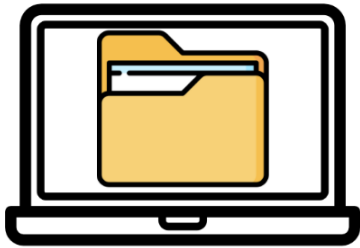
Staging Area

Repository

`program.py`

Workflow

Working Directory



program.py

index.html

style.css

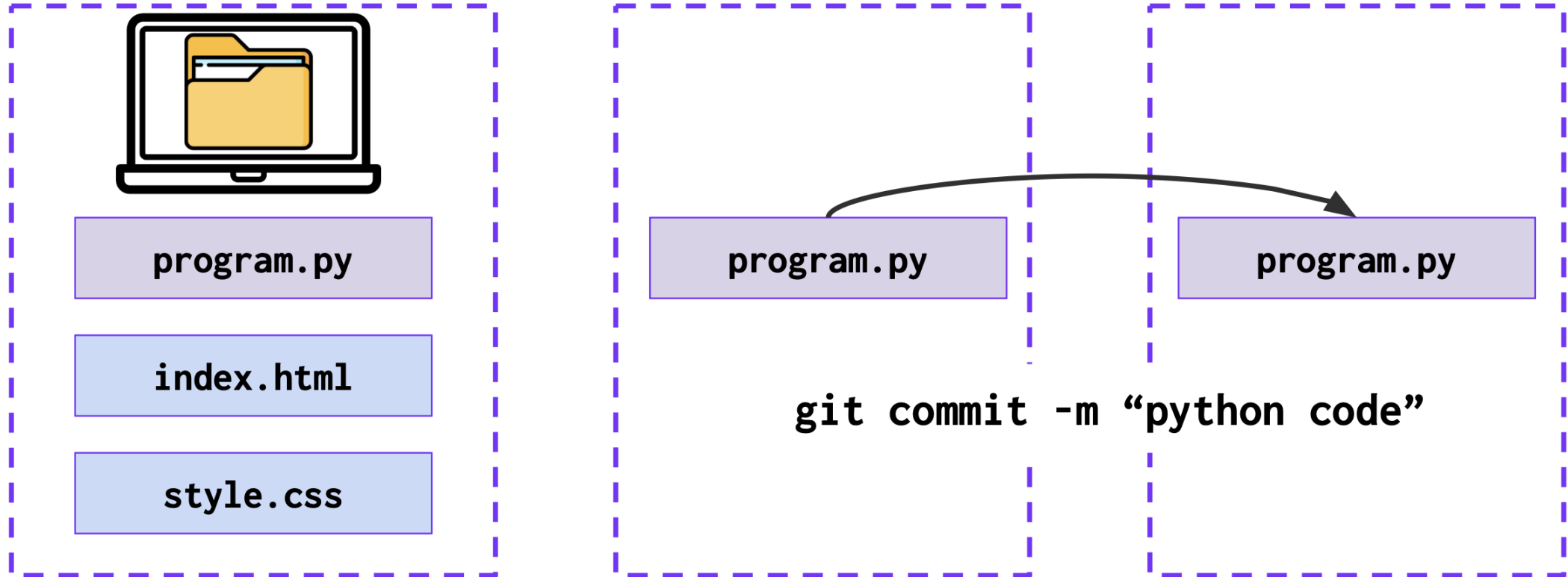
Staging Area

program.py

`git commit -m "python code"`

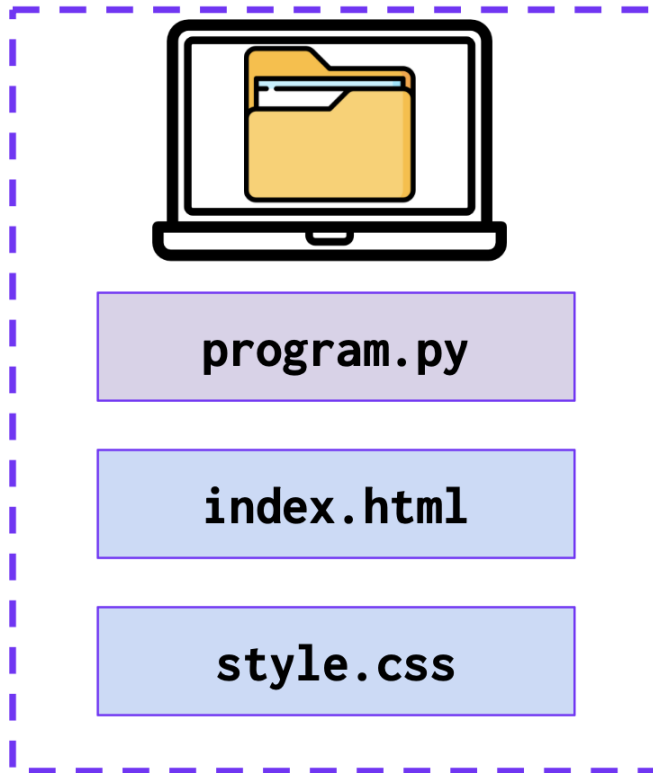
Repository

program.py



Workflow

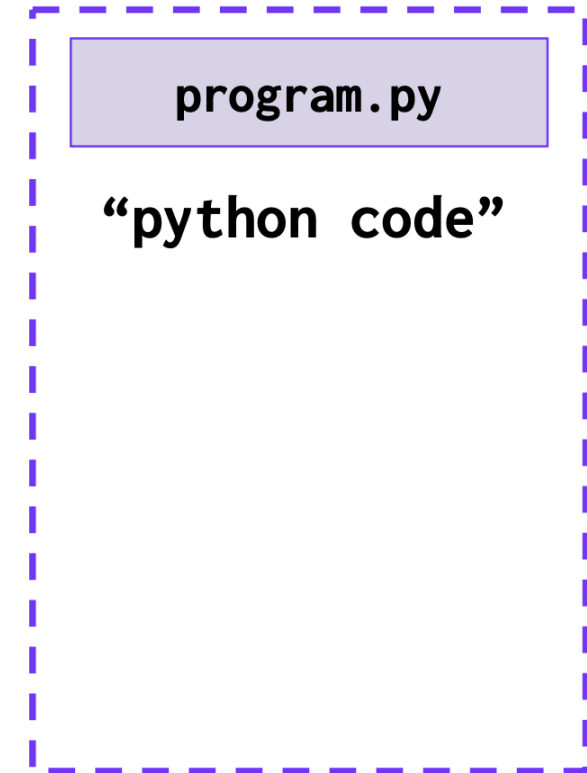
Working Directory



Staging Area

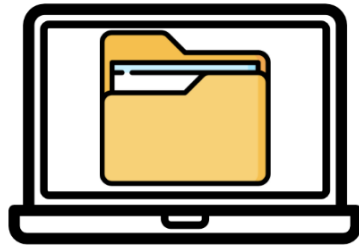


Repository



Workflow

Working Directory



`program.py`

`index.html`

`style.css`

Staging Area

`git add .`

`index.html`

`style.css`

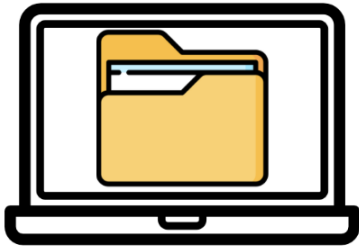
Repository

`program.py`

“python code”

Workflow

Working Directory



program.py

index.html

style.css

Staging Area

`git commit -m "site files"`

index.html

style.css

Repository

program.py

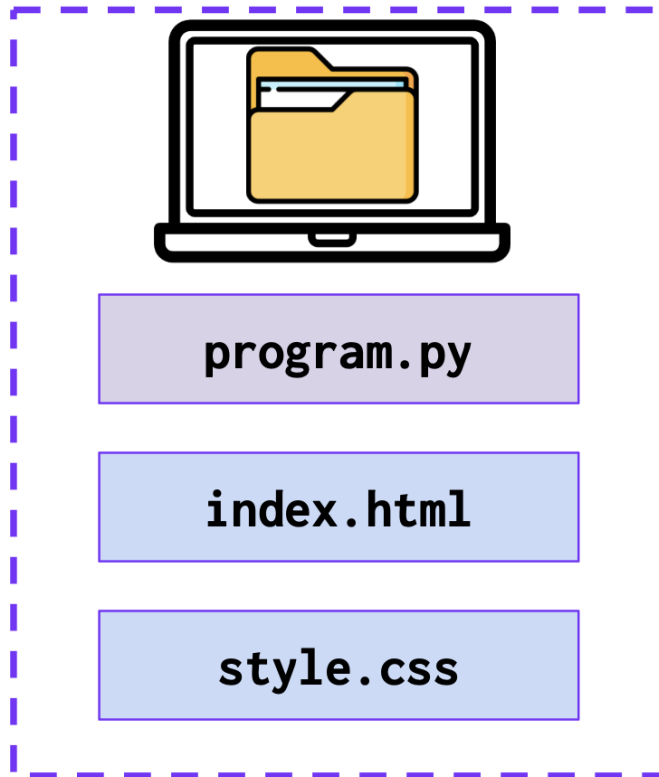
"python code"

index.html

style.css

Workflow

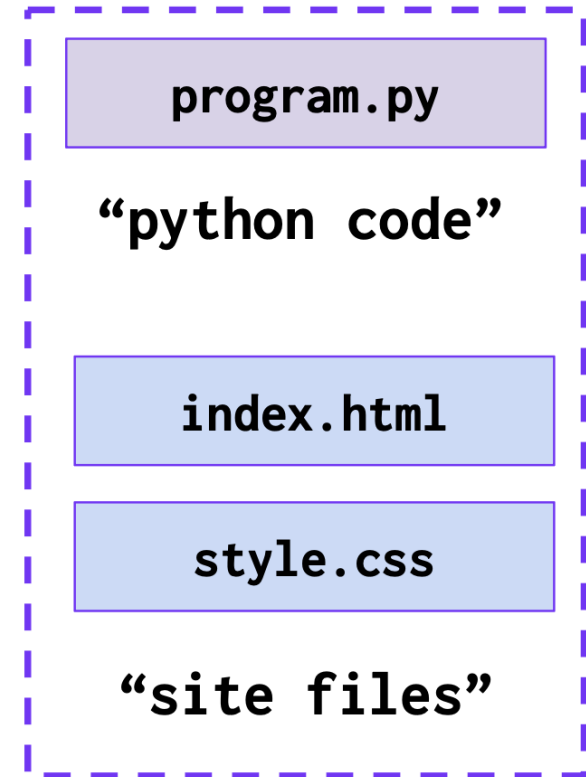
Working Directory



Staging Area

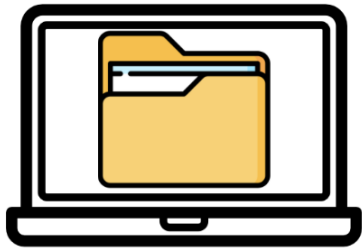


Repository



Workflow

Working Directory



`program.py`

`index.html`

`style.css`

Repository

`program.py`

“python code”

`index.html`

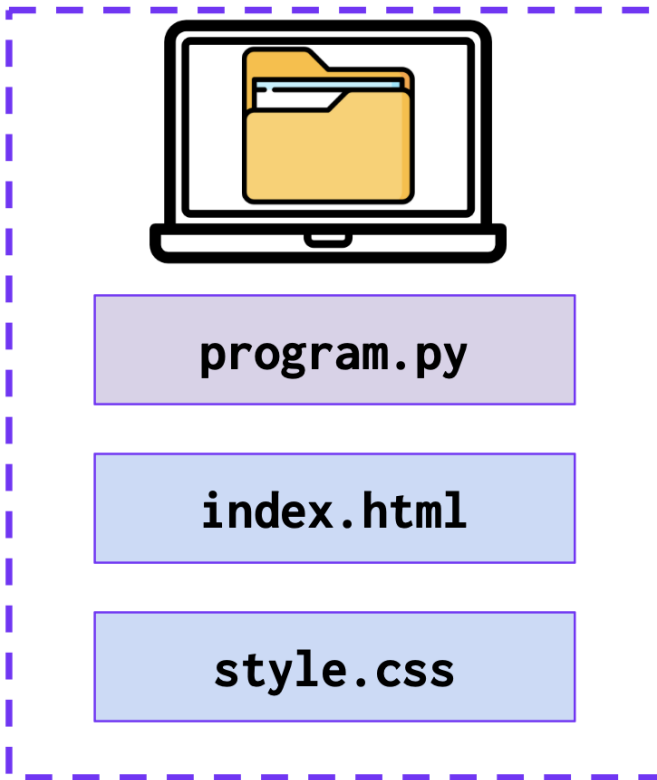
`style.css`

“site files”

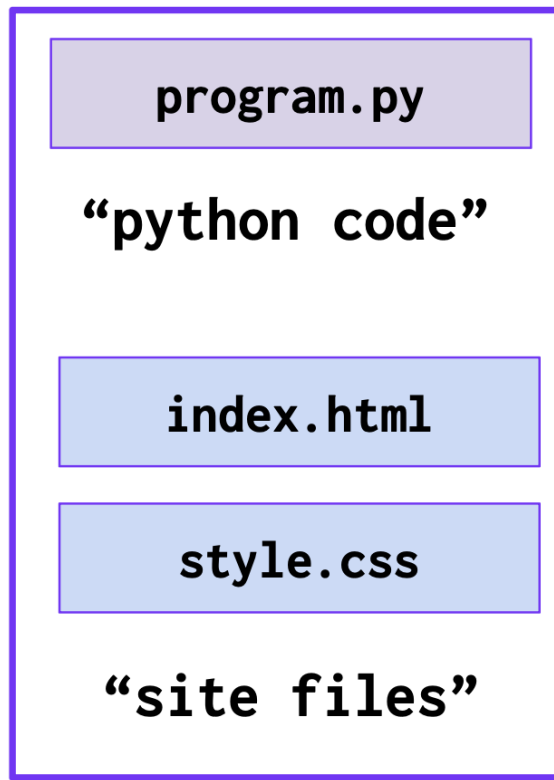


Workflow

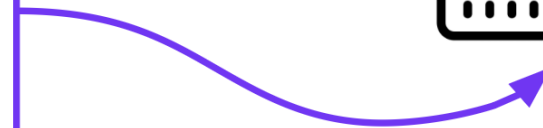
Working Directory



Repository

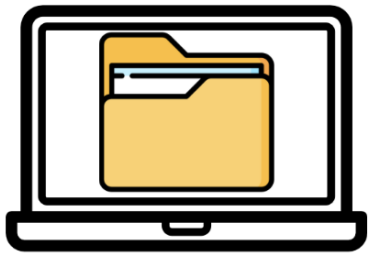


git push



Workflow

Working Directory



`program.py`

Repository

`program.py`

“python code”

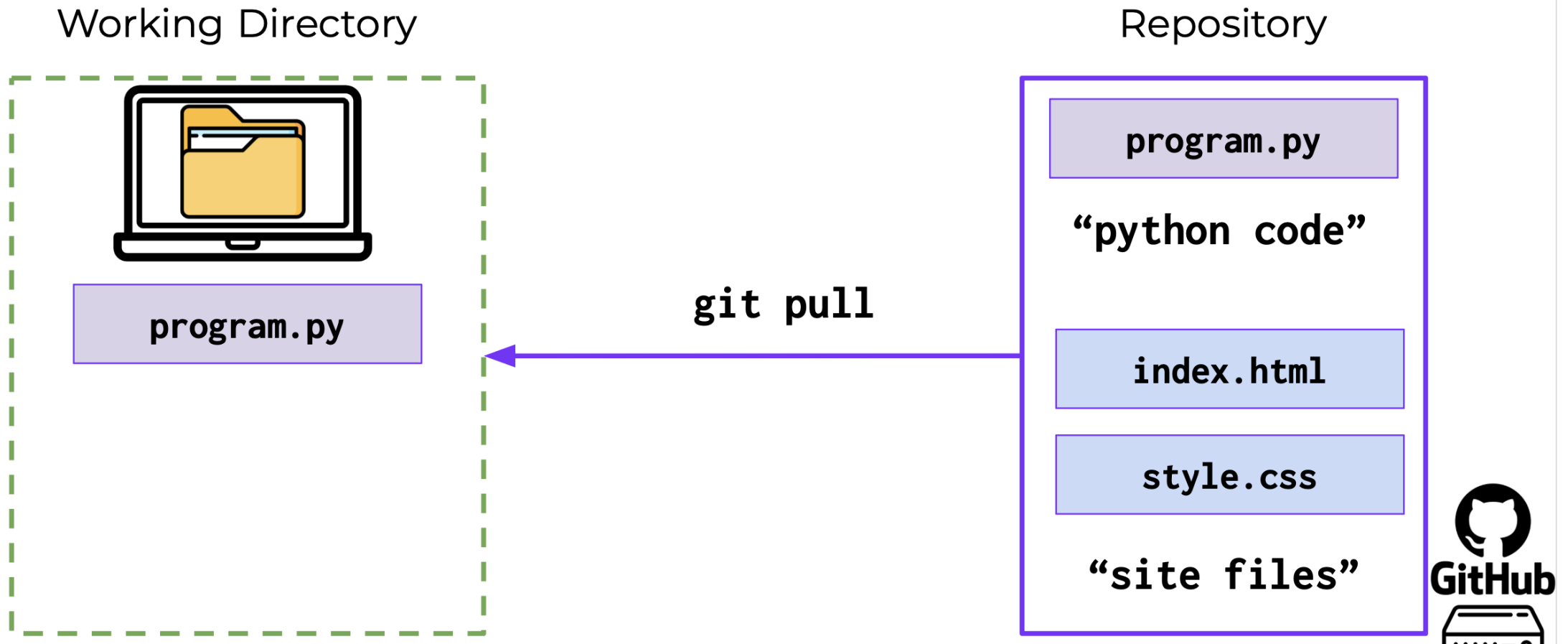
`index.html`

`style.css`

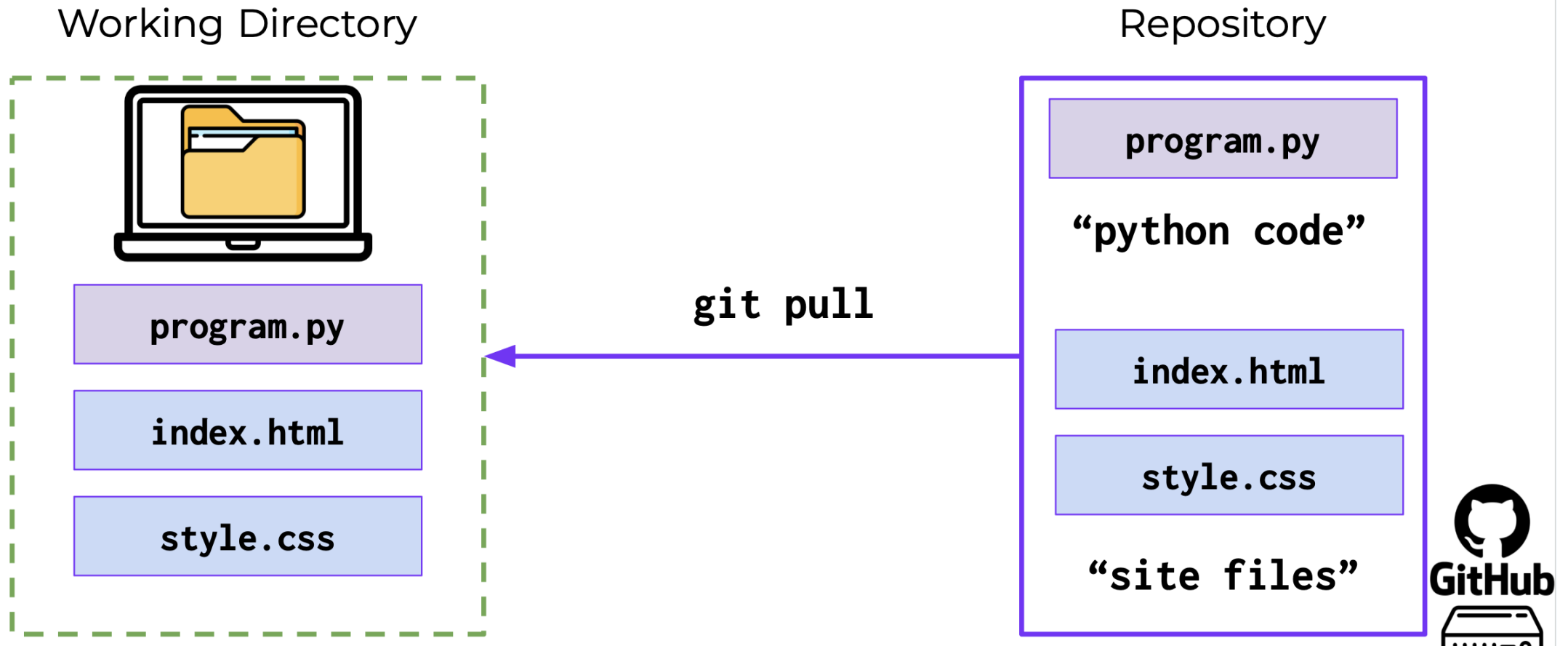
“site files”



Workflow



Workflow



Workflow

- This is the main workflow we are covering today.
- However, as we continue, try to keep in mind what would happen if we had multiple people working on different parts of the code at different times.
- How would we deal with differences or conflicting code? What about branches?

Actions

- Lets now go through the process of creating a repository, creating new files inside the working directory, adding them to the staging area, and then committing them to repository
- You will need a text editor, we recommend VS Code, Download it here. <https://code.visualstudio.com/>

Hands – on

- Git Push to remote repository

Git log

Let's explore all about
git log command



Fetch & Pull

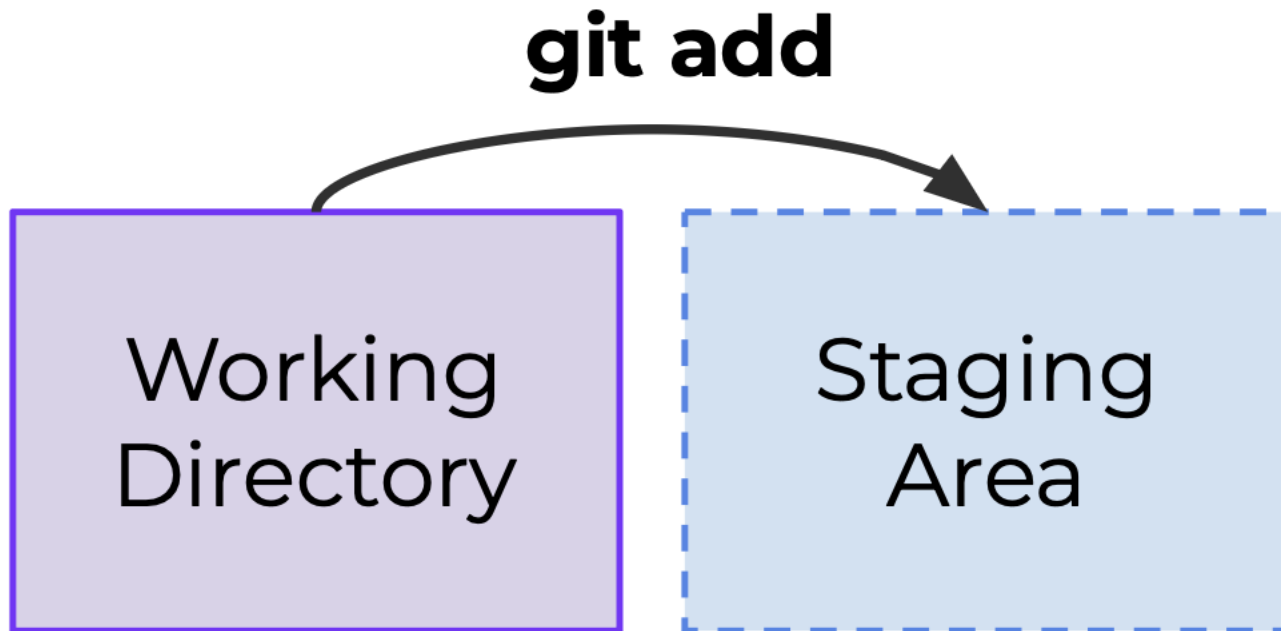
- There are two options of getting repository changes from a remote branch (like the remote branch on GitHub)
 - `git pull`
 - `git fetch`

Fetch & Pull

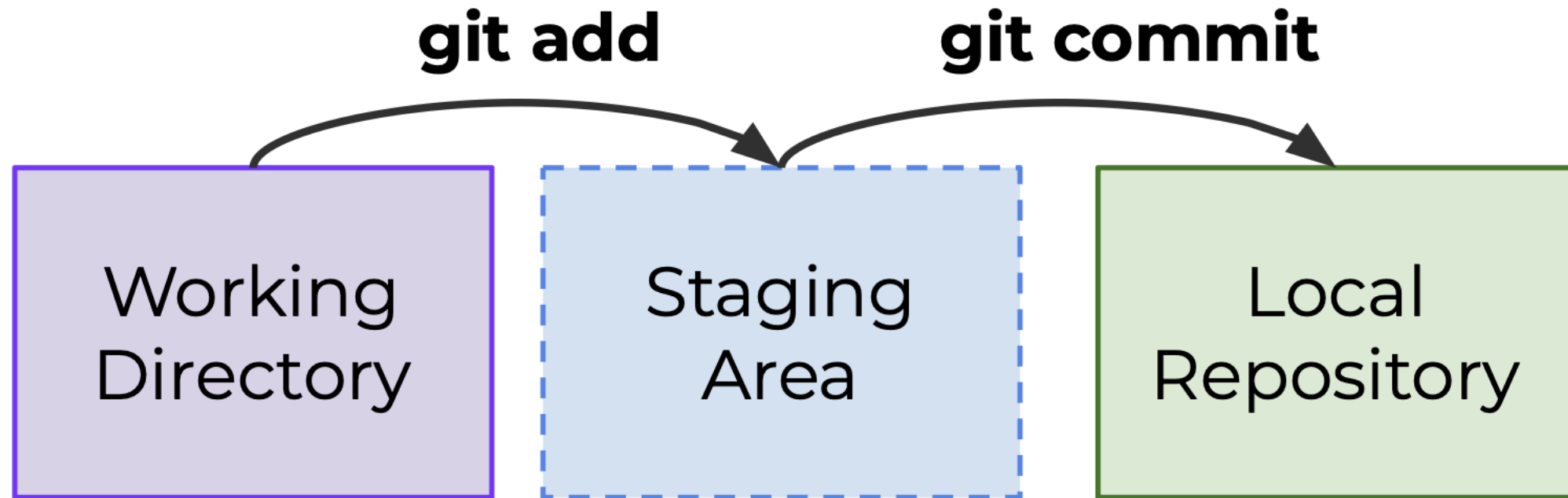


Working
Directory

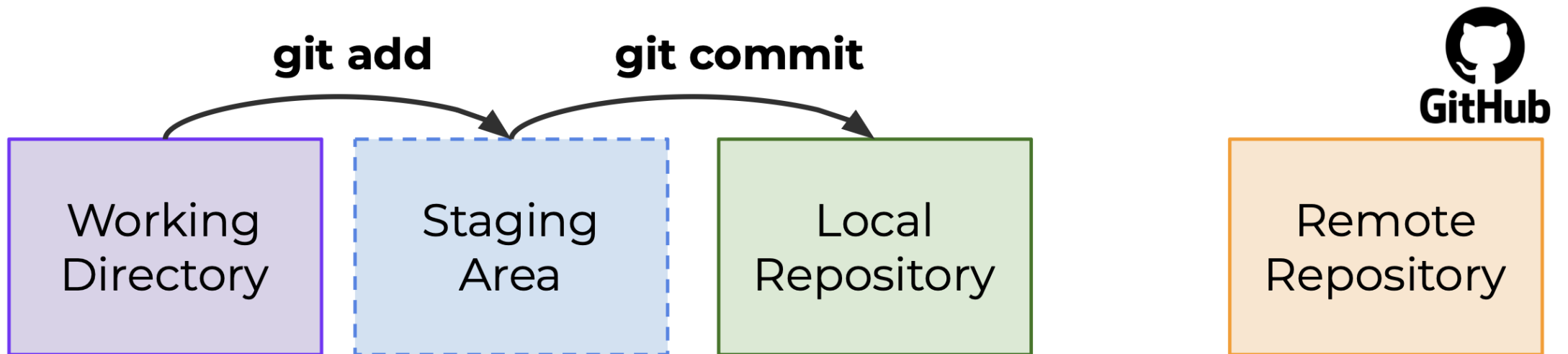
Fetch & Pull



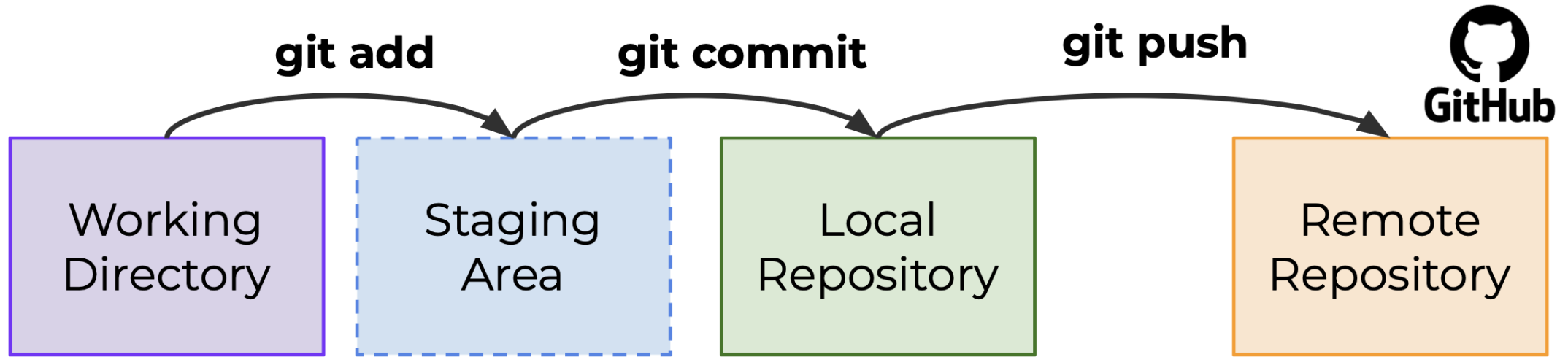
Fetch & Pull



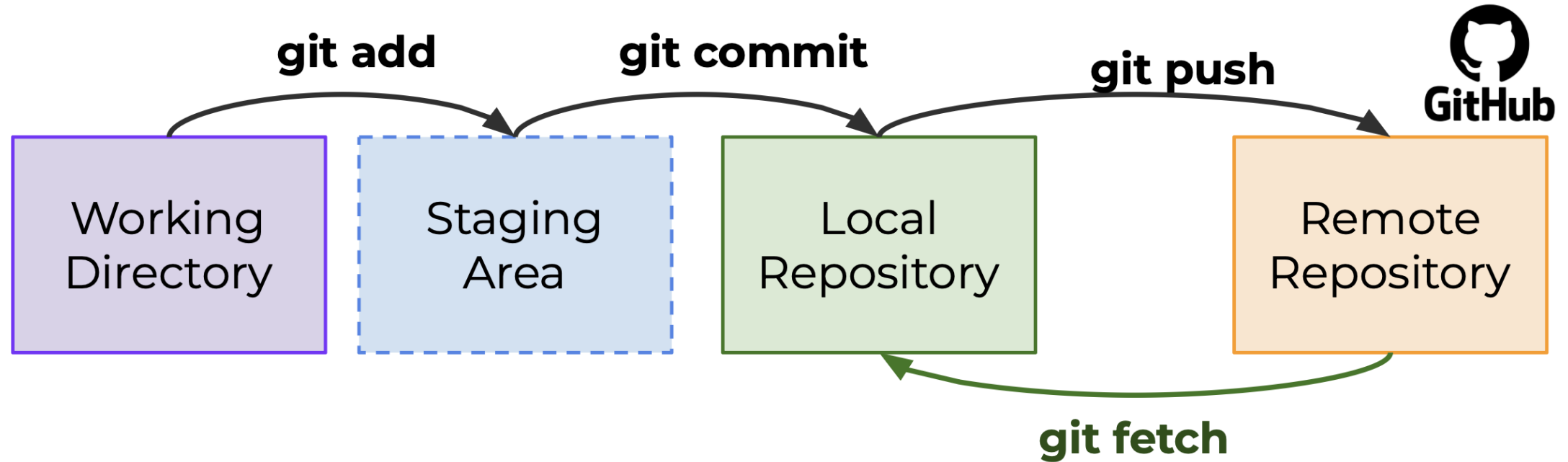
Fetch & Pull



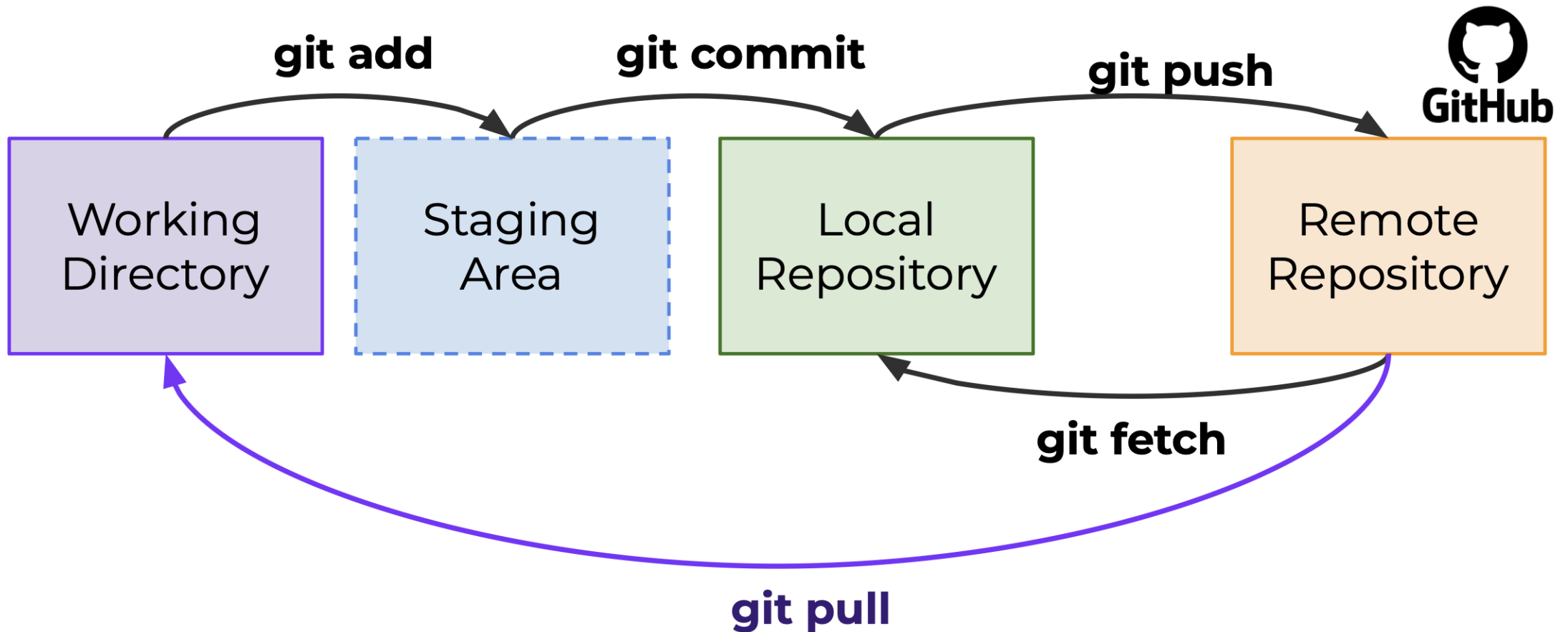
Fetch & Pull



Fetch & Pull



Fetch & Pull



Fetch & Pull

- Using git fetch will download changes from the Github remote repository, however you will not see those changes be part of the files you have in the working directory.
- Fetch allows you to grab additional work done on the remote master branch, without needing to merge it in your working directory files
- Using git fetch makes sense when you are working with others and want to see what changes they have made but arent ready to overwrite your own files yet.
- Also if you are simply working by yourself, you may want to just fetch remote changes without overwriting your latest work (later we will discover branches, which is a better way of doing this)

Fetch and Pull

- When using fetch, often you will just use:
 - **Git fetch**
- But you can specify to fetch specific branches using:
 - **git fetch <remote> <branch>**
 - **git fetch origin <branch>**

- Using “**git pull**” makes sense when you want to directly grab changes from the remote branch and directly merge them into your current branch
- This means you will literally update the files in your working directory to match up and merge with the remote branch
- If you are a solo developer working on a single master branch, you often skip using a combination of “git fetch” and “git merge” and go straight for the “git pull”
- In the next days, when we learn about branches and merging changes more carefully or stashing current changes, we’ll have a more nuanced understanding of using “git fetch” vs. “git pull”
- In general you should pull before pushing to a remote branch, to make sure you are in sync.

Tasks

- **Tasks:**
 - Create a new local repo
 - Add a text file locally
 - Create the remote repo on GitHub
 - Push the Local repo changes to GitHub
 - Create a new file on GitHub
 - Pull these changes from GitHub to the local repo

Branches and working with others

Git & GitHub

Branches and Working with Others

- Branches allow us to organize a repository and split it apart so multiple people can work on it or so a solo developer can work on different aspects of a project on a separated work.
- Master/Main Branch and Branches
- Understanding Head
- Git Branch Commands
 - Git branch, git switch, git checkout
- Delete or Rename a branch
- Merging Branches and conflicts
- Using Gif Diff
- Exercise and solution

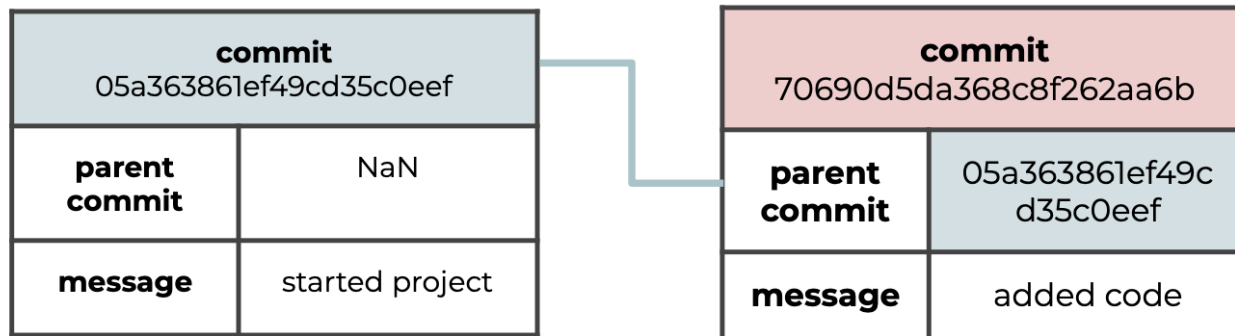
- **Commit Process**

- As we create commits, we are linking to a parent commit, showing the log of the commit history.

commit 05a363861ef49cd35c0eef	
parent commit	NaN
message	started project

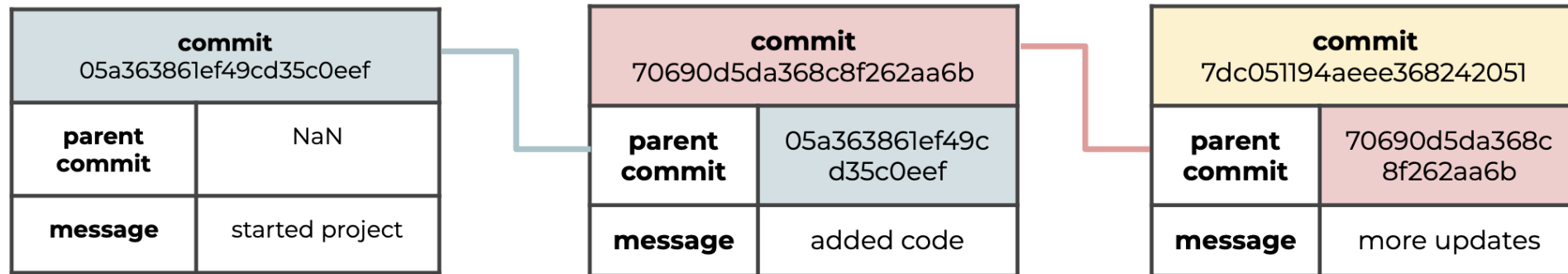
- **Commit Process**

- As we create commits, we are linking to a parent commit, showing the log of the commit history.



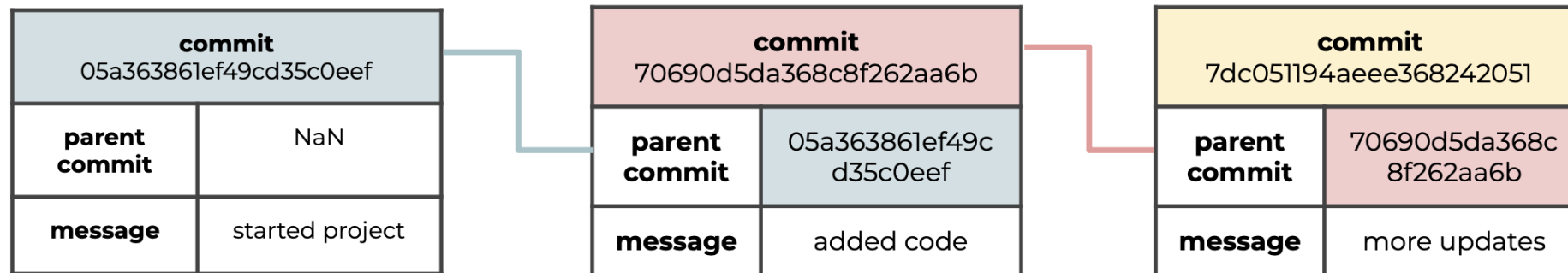
- **Commit Process**

- As we create commits, we are linking to a parent commit, showing the log of the commit history.



- **Commit Process**

- As we need incorporate the workflows of others or be able to focus on new updates without breaking old code, we need **branches**.



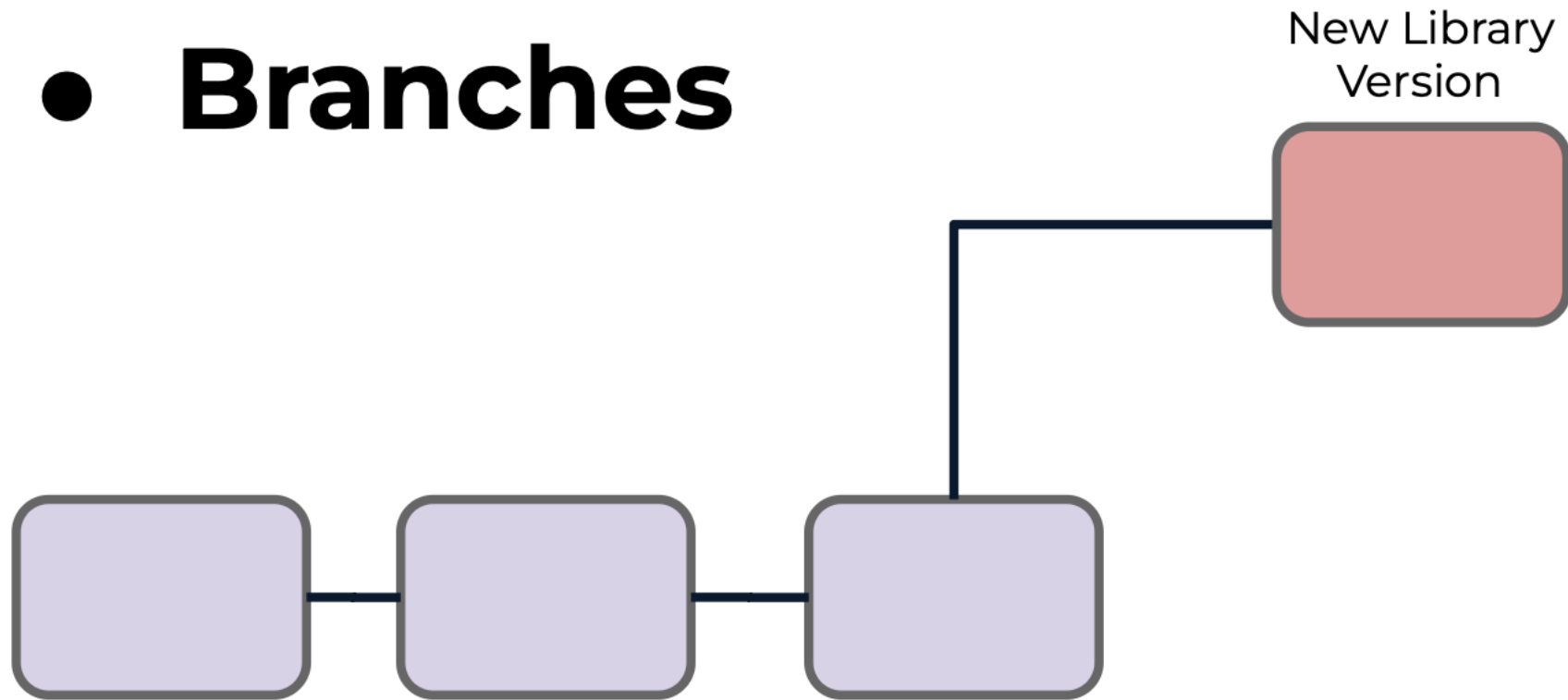
Branches

- A branch represents an Independent line of Development
- Branches serve as an abstraction for the edit/stage/commit process.
- They are a way to request a brand-new working directory, staging area and project history.
- Branches are just pointers to commits.
- When you create a branch, all git needs to do is create a new pointer, it doesn't change the repository in any other way.
- Lets explore why branches are useful for workflows.

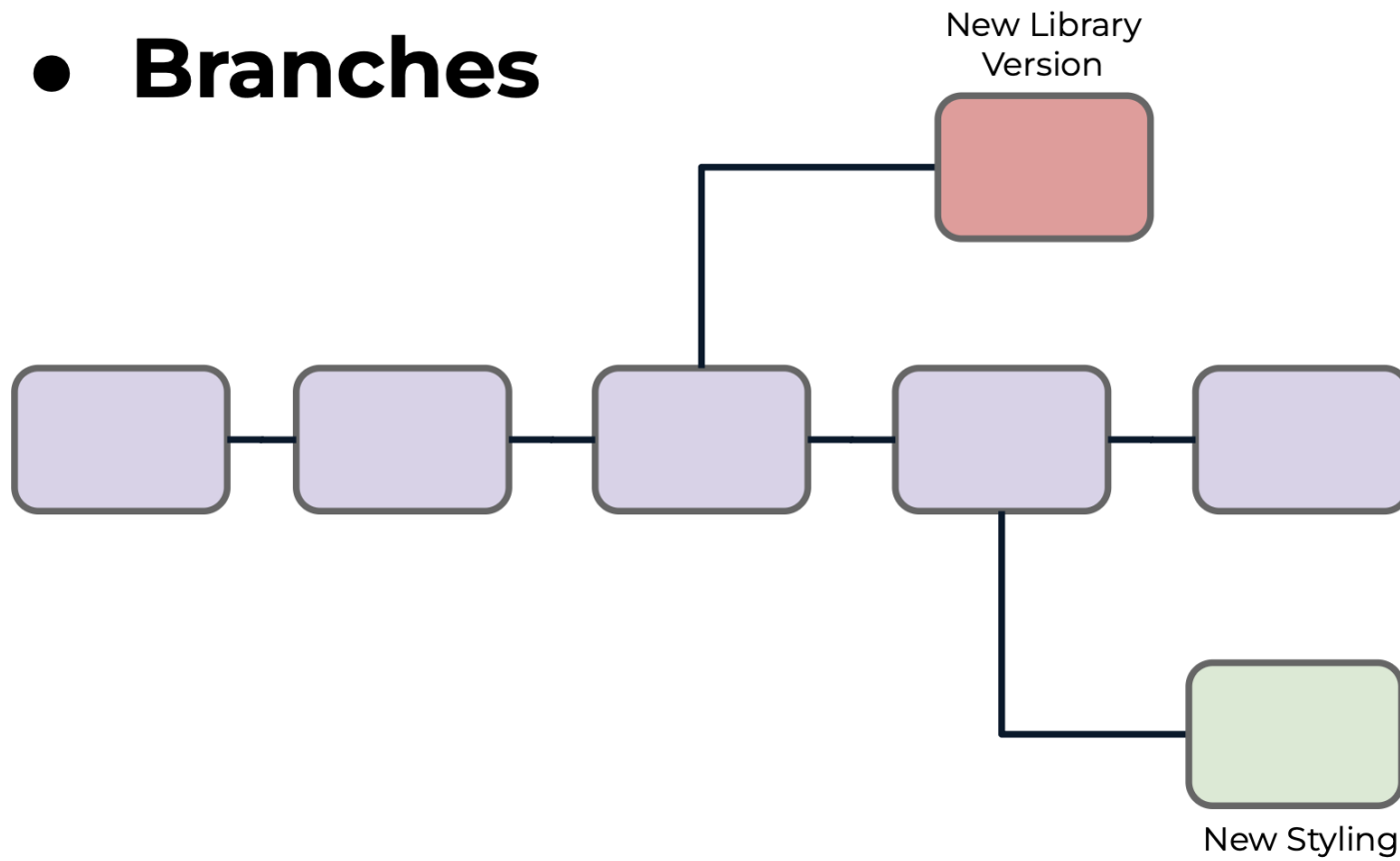
- **Branches**



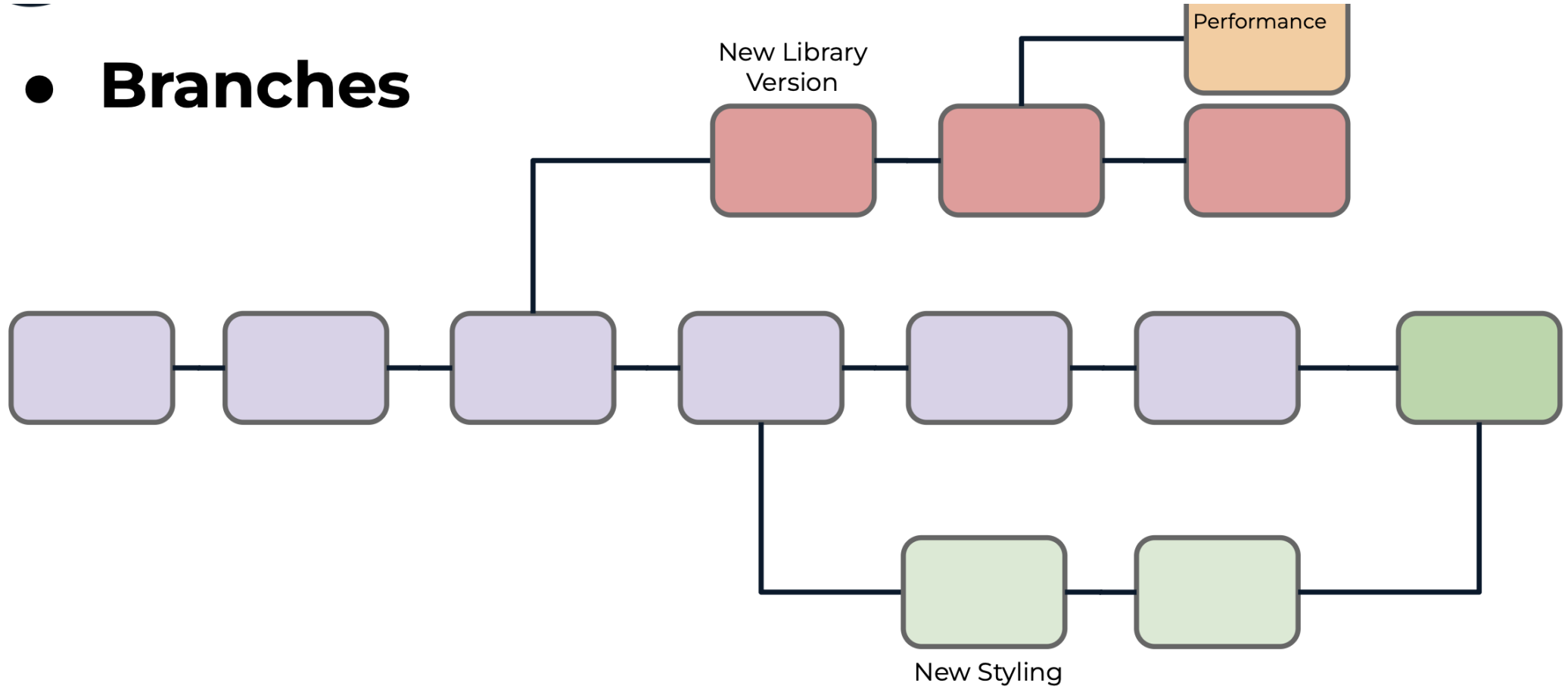
- **Branches**



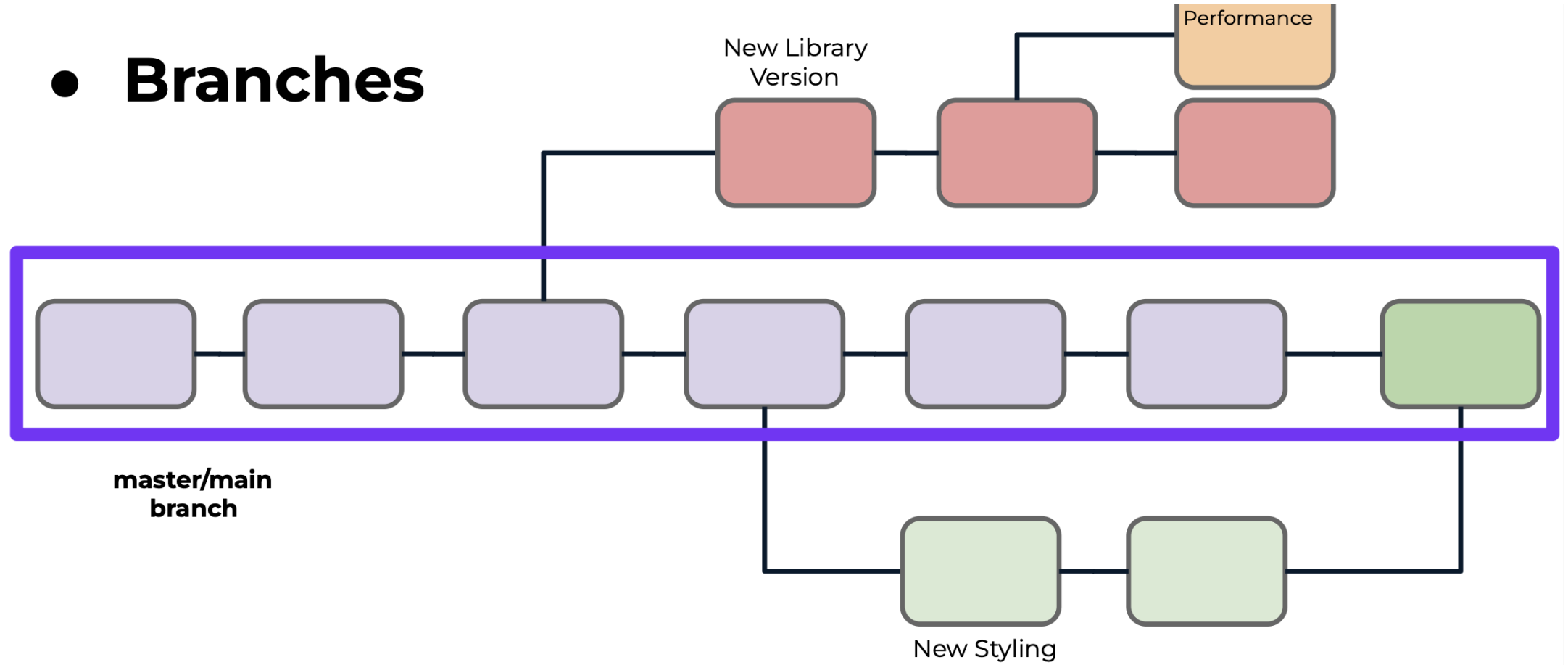
- **Branches**



- **Branches**



- **Branches**



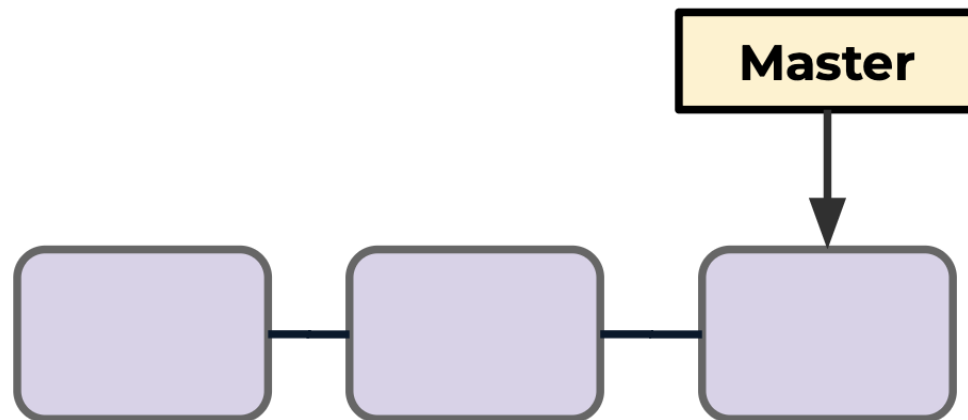
- Upon creating a new Repo with "git init"; you create a new branch called as Master/Main.
- This is a branch just like any other branch, but it simply the first one created.
 - Should code be pushed to master branch always be in working condition?
- While organizations and developers often treat this master branch as the official branch for things like deployment, this is not a requirement.
- You can use any branch for code deployment or code that's actually in use.

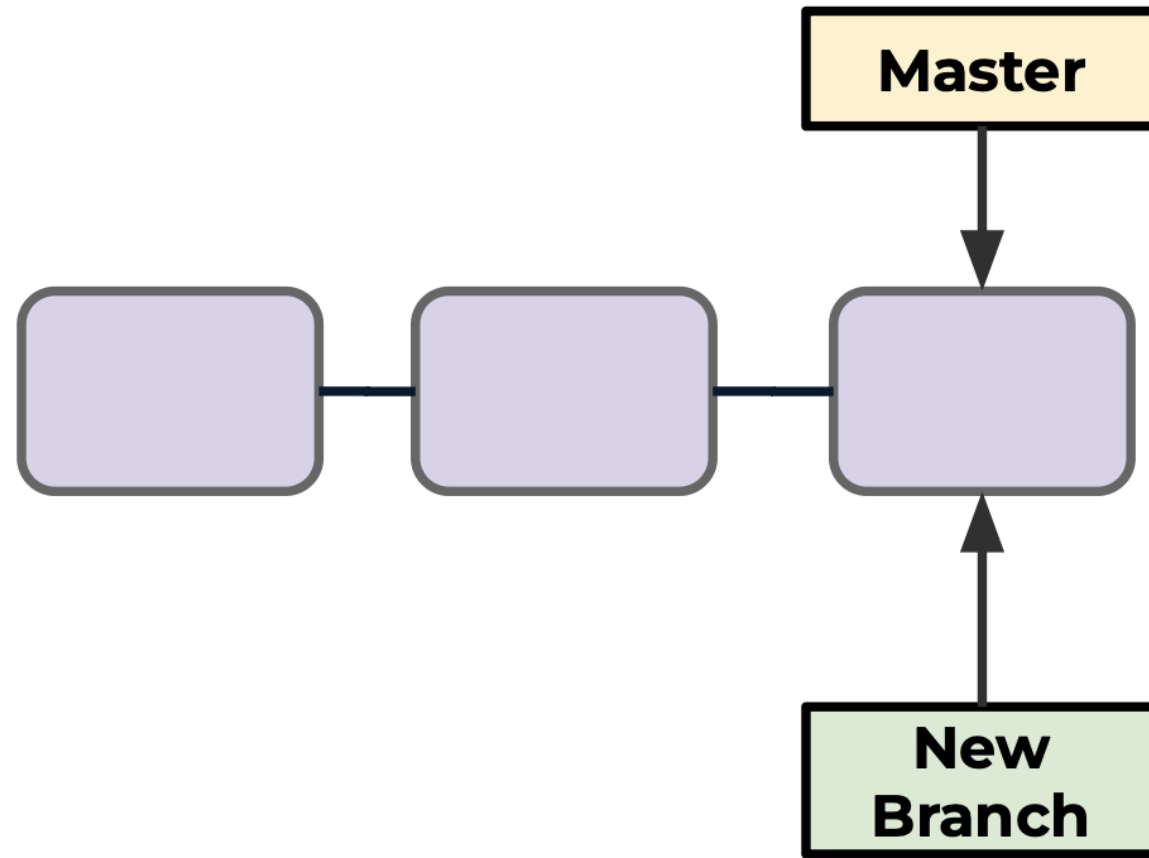
Master vs. Main

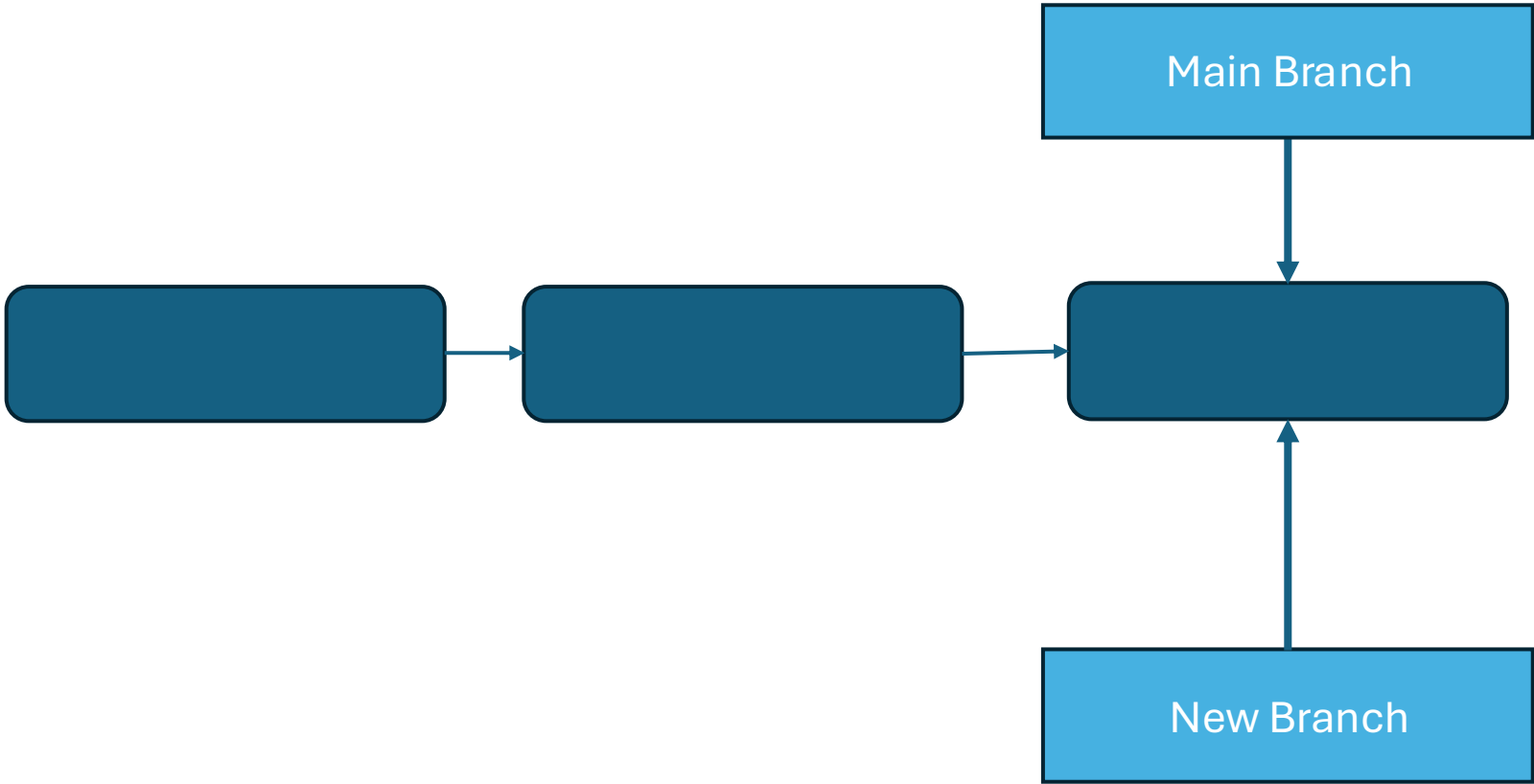
- As we discussed previously, GitHub has changed the nomenclature for this initial branch to be main branch while git is still using master branch (But this may change in future)
- You can also rename any branch (for example ; Trunk branch)

Creating a new Branch

- Before we conclude, Lets quickly go into more detail about what happens when we first create a new branch.
- Branches are just pointers to commits.
- When you create a branch, all git needs to do is create a new pointer, it doesn't change the repository in any other way.







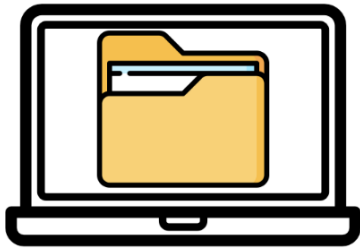
Head

Undoing Changes

Git Reset

Git Reset

Working Directory

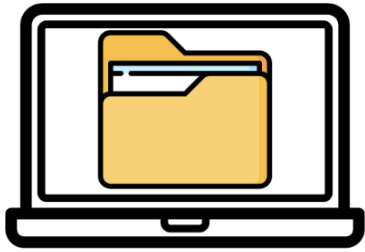


program.py

Staging Area

Repository

Working Directory



Staging Area

`program.py`

Repository

Working Directory



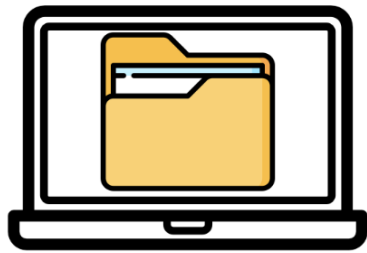
Staging Area

Repository

`program.py`

**“python code”
f3h4782**

Working Directory



`index.html`

`style.css`

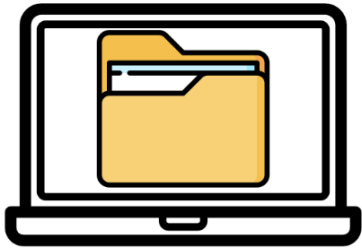
Staging Area

Repository

`program.py`

“python code”
f3h4782

Working Directory



Staging Area

`index.html`

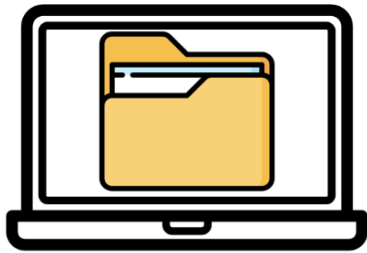
`style.css`

Repository

`program.py`

`“python code”
f3h4782`

Working Directory



Staging Area

Repository

`program.py`

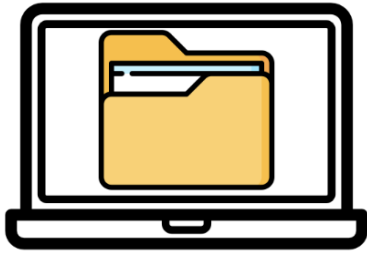
`“python code”
f3h4782`

`index.html`

`style.css`

`“style code”
g82j371`

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

index.html

style.css

“style code”
g82j371

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

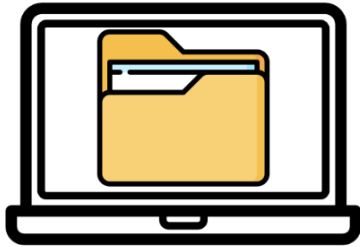
index.html

style.css

“style code”
g82j371

```
>>git reset f3h4782
```

Working Directory



`index.html`

`style.css`

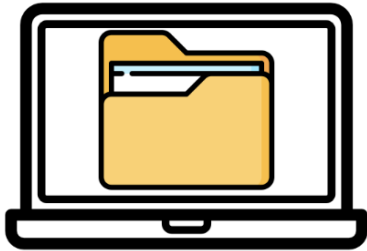
Staging Area

Repository

`program.py`

“python code”
f3h4782

Working Directory



index.html

style.css

Staging Area

Files are unchanged!
You just reset the
commits only.

Repository

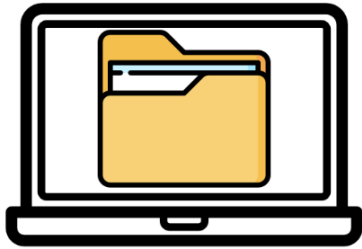
program.py

“python code”
f3h4782

- This means when you do **git reset** you won't notice a change in the files themselves, you just reset the commits.
- This is useful if you accidentally committed to the wrong branch (for example, maybe you forgot to run **git switch** right after creating a new branch, accidentally committing to the original branch).

- What if you do want the files to change?
- In the case where you just want to undo everything, including changes and have the branch files look like they did at a previous commit, you add the flag **--hard**.
- For example:
 - **git reset f3h4782 --hard**

Working Directory



Staging Area

Repository

`program.py`

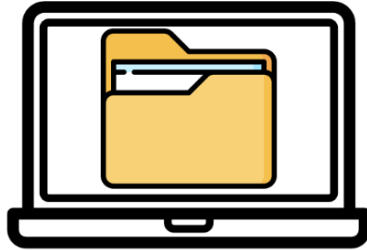
“python code”
f3h4782

`index.html`

`style.css`

“style code”
g82j371

Working Directory



Staging Area

Repository

program.py

“python code”
f3h4782

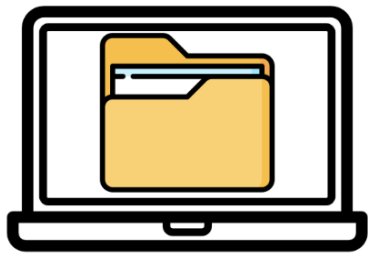
index.html

style.css

“style code”
g82j371

```
>>git reset f3h4782 --hard
```

Working Directory



Staging Area

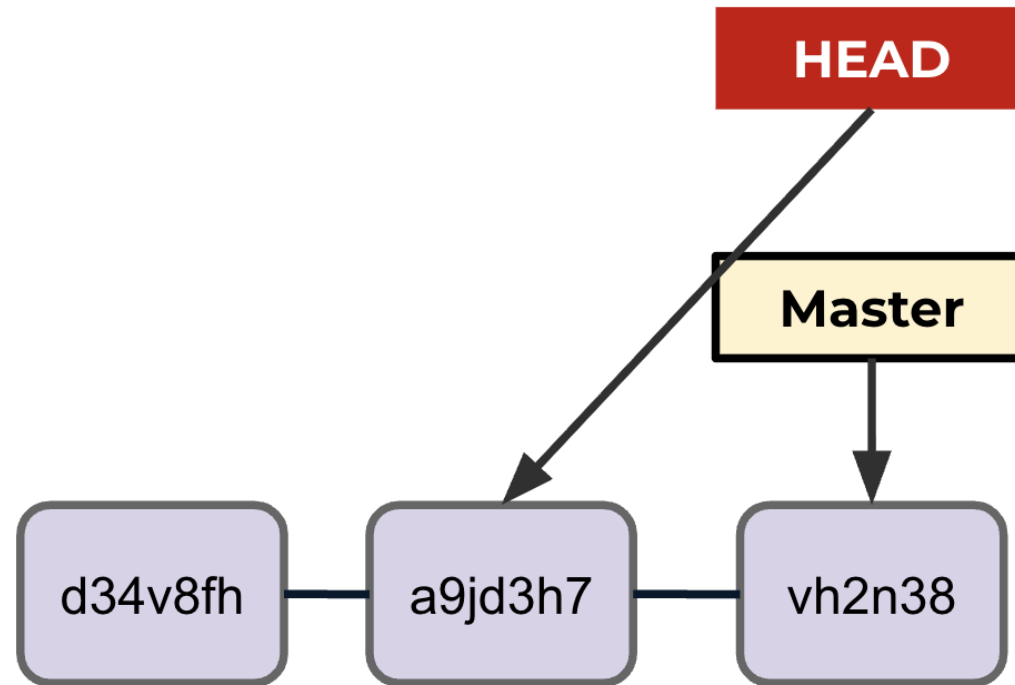
Repository

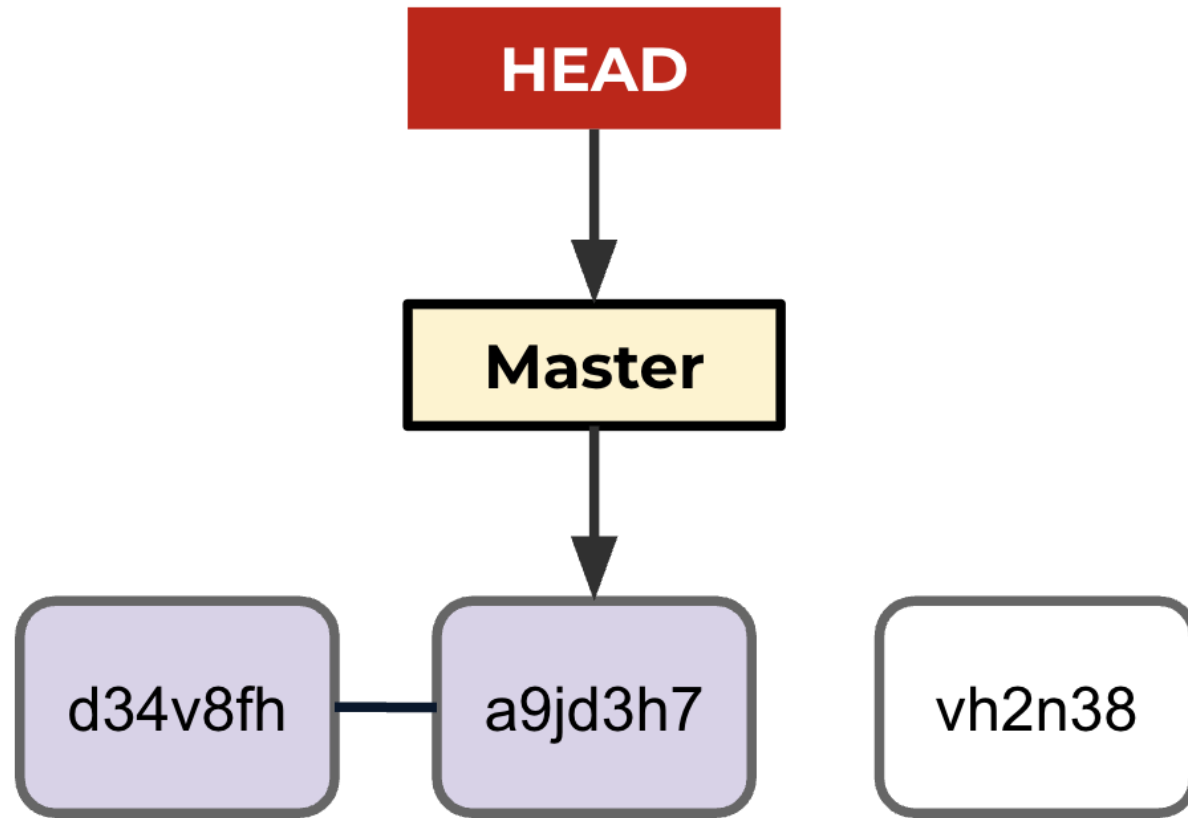
program.py

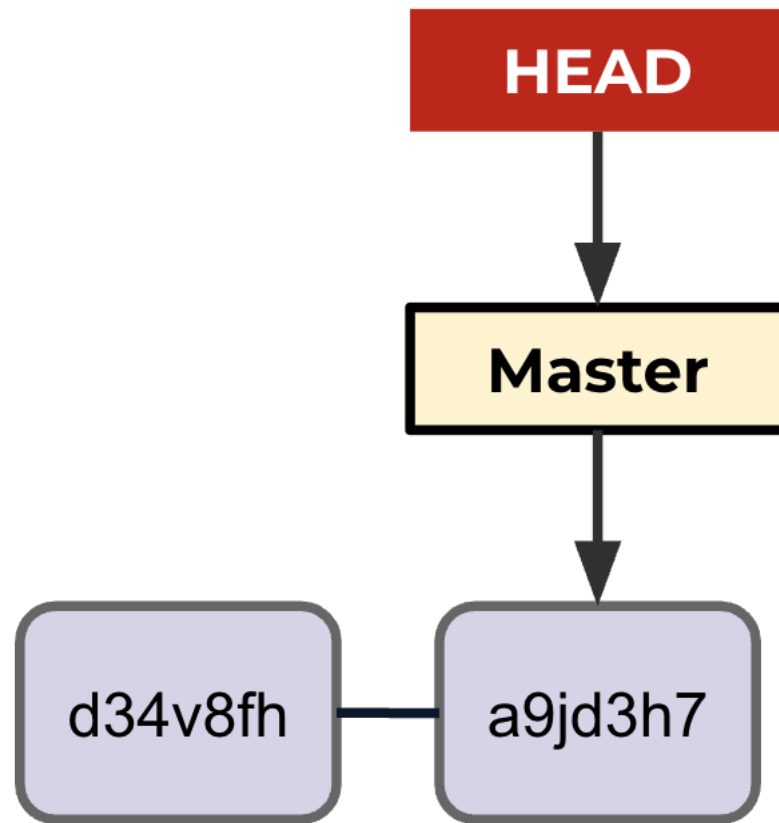
“python code”
f3h4782

```
>>git reset f3h4782 --hard
```

- We can visualize a **git reset** moving back to a previous commit, but not undoing file changes (unless it is --hard)







1. Branches
2. Default Branch (git init) – Main Branch
3. Git clone – Remote
4. Commands related to Git

